AETHER

Aether por dentro

El libro completo del proyecto

Borrador de revisión · Parte B (hardware y mecánica)
24 de octubre de 2026

Índice

Capítulo 2: Electricidad y física para mecatrónicos 3

Capítulo 3: Estructura y mecánica del robot y del carrito 10

Capítulo 4: Alimentación, baterías, protecciones y paro de emergencia 18

Capítulo 5: Motores, drivers y el ESP32 25

Capítulo 6: Sensores y cámaras 32

Capítulo 7: La celda de manufactura 38

Capítulo 8: La Raspberry Pi 4 y la seguridad eléctrica del GPIO 46

Capítulo 9: El Jetson Orin Nano Super 52

Glosario de siglas y términos 59

Pendientes de confirmar con el equipo 63

Capítulo 2: Electricidad y física para mecatrónicos

Voltios, amperios, vatios, tierra común, torque y fricción: lo mínimo que hay que entender para que Aether se mueva sin quemar nada ni a nadie.

Una frase de tu propio repositorio que vale cuatro semestres

En la guía de construcción de la celda hay una línea que parece un detalle y es una clase completa de electrónica: "*Un GPIO de la Pi es de 3,3 V: 24 V lo quema.*" Dos oraciones más abajo, otra: "*Un módulo infrarrojo de 5 V de Arduino no sirve en esta entrada.*"

Las dos dicen lo mismo con distinto número: **cada cosa eléctrica aguanta un voltaje, y si le das más, se rompe; si le das menos, no entiende lo que le dices**. En Aether conviven tres mundos de voltaje: 3.3 V (los pines de la Raspberry Pi y del ESP32), 5 V (la alimentación de varias placas) y 24 V (la parte industrial de la celda). Este capítulo te enseña a moverte entre ellos sin chamuscar nada, y después añade la mitad mecánica: qué fuerza hace falta para mover un robot de hasta 40 kg.

Una aclaración de método antes de empezar. Este capítulo es **teoría con ejemplos tuyos**. Las cifras de ejemplo que uses para entender (un LED, una rampa) están marcadas como ejemplo; los datos reales del hardware que todavía no están decididos (motores, llantas, peso final) están marcados **PENDIENTE** y no se inventan.

Voltios, amperios y vatios con una manguera

Imagina una manguera de jardín.

- El **voltaje** (símbolo V, unidad: voltio) es la **presión** del agua. Es lo que "empuja" a la electricidad por el cable. Una batería de 12 V empuja con 12 de presión.
- La **corriente** (símbolo I, unidad: amperio, A) es **cuánta agua pasa por segundo**. Una corriente grande es una manguera ancha a todo chorro.
- La **potencia** (símbolo P, unidad: vatio, W) es **cuánto trabajo hace** ese chorro. Se calcula multiplicando: **P = V × I**.

Dos ejemplos de tu proyecto. El Jetson Orin Nano Super tiene modos de energía de 7 W, 15 W, 25 W y MAXN_SUPER (el máximo); en la bitácora del 21 de octubre quedó anotado que **hoy corre en el modo de 25 W** (**VERIFICADO**, salida real de `nvpmodel` pegada por

ti). Eso significa que, cuando trabaja fuerte, le pide a su fuente unos 25 vatios. Y la fuente oficial de la Raspberry Pi 5 que se llegó a considerar era de 27 W a 5 V, es decir, hasta unos 5.4 amperios.

Por qué importa: **los cables, los fusibles, los conectores y las fuentes se eligen por amperios**, no por voltios. Un cable delgado que aguanta 1 A se calienta como resistencia de tostador si le pides 10 A.

La ley de Ohm, o por qué un LED necesita su resistencia

La **ley de Ohm** dice que **V = I × R**, donde R es la **resistencia** (unidad: ohmio, símbolo Ω), algo que se opone al paso de la corriente. Es como un estrechamiento en la manguera.

Ejemplo clásico, que además es lo que tu plan pide hacer antes de conectar una carga real a la Raspberry (ver el capítulo 8): encender un **LED** (diodo emisor de luz, *Light-Emitting Diode*). Un LED es traicionero: no se opone casi nada a la corriente, así que si lo conectas directo a un pin se "come" toda la corriente que pueda y se quema. Se le pone una resistencia en serie que limite la corriente.

> **Ejemplo de cuenta (regla general, no un dato de tu proyecto):** un pin de 3.3 V, un LED rojo que consume unos 2.0 V cuando enciende y que queremos a 10 mA (0.010 A). La resistencia sobrante debe "gastar" 3.3 − 2.0 = 1.3 V. R = 1.3 / 0.010 = **130 Ω**. Se usa la resistencia comercial más cercana por arriba, 150 Ω, que deja pasar unos 8.7 mA.

Observa lo útil que es: con una sola fórmula calculaste algo que protege un componente de 5 centavos. La misma lógica, a otra escala, explica por qué entre el pin de una Pi y un motor hay un **driver** o un **relé** en medio (capítulos 5 y 8).

Corriente continua y alterna: de qué lado vives

Hay dos tipos de electricidad. La **corriente continua** (CC o DC, *Direct Current*) fluye siempre en el mismo sentido: es la de las baterías, las fuentes de las computadoras y todo lo que hay dentro de tu robot y de la parte de control de tu celda. La **corriente alterna** (CA o AC, *Alternating Current*) cambia de sentido muchas veces por segundo: es la del enchufe de la pared. Casi todo lo que vas a cablear en Aether es continua (12 V de batería,

5 V, 3.3 V, 24 V del tablero). **La alterna aparece solo en el enchufe** que alimenta la fuente. Regla de oro general: no abras ni cables nada conectado directo a la pared sin que lo revise la persona que sabe de eso en tu equipo.

Tierra común: el cable que nadie dibuja

Este es el tema que más confunde, y en la guía de construcción de tu proyecto está explicado en una frase: *"Los 0 V de la batería de motores y el GND de la Pi se unen en un solo punto."*

GND (*ground*, en español "tierra" o "referencia de 0 V") es el punto contra el que se miden todos los voltajes. Un voltaje no existe solo: siempre es una **diferencia entre dos puntos**. Cuando una placa le manda a otra una señal de "3.3 V", lo que realmente le dice es "estoy 3.3 V por encima de mi referencia". Si las dos placas no comparten referencia, cada una tiene su propia idea de dónde está el cero, y la señal llega como ruido.

Por eso el ESP32, el driver de motores y la computadora principal **comparten GND**. Y por eso importa *dónde* se unen: la guía pide unir los negativos **en un solo punto** (el GND del driver). Si los unes en varios lugares al azar, las corrientes grandes de los motores pueden meterse por los cables delgados de señal y generar fallas raras e intermitentes, de esas que "solo pasan los jueves".

Niveles lógicos: 3.3 V, 5 V y 24 V no se entienden entre sí

Una señal digital es un cable que está "alto" (casi el voltaje de la placa) o "bajo" (casi 0 V). Eso se llama **nivel lógico**. En tu proyecto:

Dispositivo	Nivel de sus pines	Qué pasa si le mandas más
Raspberry Pi (GPIO)	3.3 V	Más de 3.3 V puede dañar el pin o el chip
ESP32	3.3 V	Igual; el ESP32 de la guía se alimenta por USB, pero sus pines son de 3.3 V
Sensores industriales de la celda	24 V	No se conectan a pines de la Pi
Módulos "de Arduino" de 5 V	5 V	No sirven en una entrada de 24 V ni deben meterse a un pin de 3.3 V

(Esa tabla combina lo que dicen la guía de construcción y la hoja de conexión de la celda con la regla general de cada familia de placas.)

La solución industrial a ese problema es el **aislamiento**: un **relé** (interruptor mecánico que se cierra con una bobina) o un **optoacoplador** (un LED y un fototransistor dentro de un chip, que pasa la señal con luz y no con cobre). Así el mundo de 3.3 V *manda* sin *tocar* el mundo de 24 V. Esa es exactamente la razón de que, en la celda, la Raspberry no esté cableada a los sensores y que haya un relevador de por medio (ver capítulo 7).

PNP y NPN: el problema de los tres hilos

Los **infrarrojos industriales** de la celda tienen tres hilos: **marrón** (+24 V), **azul** (0 V) y **negro** (la señal). Pero el hilo negro puede comportarse de dos maneras:

- **PNP**: cuando detecta, su salida **entrega** +24 V.
- **NPN**: cuando detecta, su salida **conecta a 0 V**.

La entrada del controlador (PLC o LOGO) tiene que ser del tipo compatible. Si no coinciden, **el LED de la entrada nunca se enciende aunque el sensor sí detecte**, y no hay forma de arreglarlo en programa. La hoja de la celda lo dice en una línea clara: "*Eso no se arregla en Python.*" Moraleja de ingeniero: **antes de escribir una sola línea de código, comprueba con la mano y el LED que la entrada ve al sensor.** Qué sensores y qué controlador son los reales está **PENDIENTE (P-14)**.

Fusibles: el eslabón que se sacrifica a propósito

Un **fusible** es un hilito calibrado que se funde cuando pasa más corriente de la permitida y corta el circuito. Es barato y está ahí para que lo que se queme sea él y no el motor, la batería o el cable. Regla general: el fusible se elige **más grande que la corriente normal del circuito y más chico que lo que aguantan los cables y componentes**. En el diseño del carrito que vive en el repositorio (la hoja de armado, capítulo 3) hay dos ramas con fusible, de 15 A y de 10 A; en el capítulo 4 las vemos con calma.

De la electricidad al movimiento: fuerza, torque y rodar

Ahora la mitad mecánica, porque un robot es la electricidad volviéndose empuje.

- **Fuerza** (F, unidad: **newton**, N): lo que empuja o jala. El **peso** es la fuerza con que la Tierra jala una masa: **peso = masa × g**, con g ≈ 9.81 m/s². Un robot de 40 kg pesa unos **392 N**.

- **Torque** o **par** (símbolo τ, unidad: **newton-metro**, N·m): la fuerza que hace *girar*. Es la fuerza multiplicada por la distancia al eje: **τ = F × r**. Es lo que sientes al apretar una tuerca con una llave larga: la misma fuerza con una llave más larga da más torque.
- **Fricción de rodamiento**: lo que frena a una llanta que rueda sobre el piso. Se aproxima como **F = coeficiente × peso**. Para una llanta de caucho sobre piso liso, un coeficiente típico (regla general) está entre 0.02 y 0.05.
- **Tracción**: la fuerza máxima que la llanta puede transmitir al piso **sin patinar**: **coeficiente de agarre × peso que carga esa llanta**. Para caucho sobre piso liso un valor típico es del orden de 0.5 a 0.8 (regla general).
- **Pendiente**: subir una rampa de ángulo α añade una fuerza **peso × sen(α)** en contra.

Un ejemplo con los números que sí tienes

Mario definió estos datos de partida del robot (todos **PENDIENTE de confirmar**, son estimaciones, no medidas; la velocidad máxima de 0.35 m/s y la aceleración de 0.4 m/s² salen de la hoja de armado): peso total máximo **40 kg** y unos **17 kg sobre cada llanta motriz**; el robot es **diferencial** (dos motores, una llanta por motor). El diámetro de la llanta **PENDIENTE (P-03)**: para poder hacer la cuenta usaremos 102 mm de diámetro (radio 0.051 m), que es el que dibuja la hoja de armado, **solo como ejemplo**.

> **Ejemplo de cuenta con datos estimados (NO es un dato confirmado de tu robot):**
>
> - Peso: 40 × 9.81 ≈ **392 N**.
> - Fricción de rodamiento con coeficiente 0.03 (supuesto): 0.03 × 392 ≈ **11.8 N**.
> - Para acelerar a 0.4 m/s² (el límite de aceleración que anota la hoja de armado para el viaje): 40 × 0.4 = **16 N**.
> - Fuerza total en piso plano: 11.8 + 16 ≈ **27.8 N** entre las dos llantas, o **~13.9 N por llanta**.
> - Torque por llanta con radio 0.051 m: 13.9 × 0.051 ≈ **0.7 N·m**.
> - Ahora una rampa de **5°** (supuesto): 392 × sen(5°) ≈ **34 N** extra. Total ≈ 62 N, ~31 N por llanta, torque ≈ **1.6 N·m** por llanta.
> - ¿Patinan? Cada llanta carga 17 kg (≈ 167 N). Con agarre 0.6: 0.6 × 167 ≈ **100 N** disponibles por llanta; los 31 N necesarios caben bien.
> - Potencia mecánica al acelerar a la velocidad máxima de la hoja (0.35 m/s) en piso plano: 27.8 N × 0.35 m/s ≈ **10 W**, sin contar pérdidas del motor ni de la transmisión. A velocidad constante, solo la fricción: 11.8 × 0.35 ≈ 4 W.

Dos lecciones que valen más que los números. **Primera**: la rampa pesa más que todo lo demás; por eso la pendiente más inclinada que va a encontrar el robot es *la* pregunta que decide el motor (qué pendientes hay en el lugar donde va a trabajar está **PENDIENTE (P-02)**). **Segunda**: la cuenta de torque necesaria es lo primero que se hace antes de comprar motores, no después. Con ese torque, un ingeniero busca un motor (y un reductor, si hace falta) que lo entregue **con holgura**, normalmente con un margen de seguridad del doble (regla general).

Cómo medir sin miedo: el multímetro

El **multímetro** es la herramienta que convierte "creo que" en "mido que". Tres usos que vas a necesitar:

1. **Continuidad** (el modo con símbolo de bocina): comprueba que dos puntos están unidos por cable. En la hoja de armado, antes de conectar la batería, se pide mirar continuidad entre el positivo y el negativo: **si pita, hay un corto** y no se conecta nada.
2. **Voltaje** (V con una raya recta, para continua): se mide **en paralelo**, poniendo las puntas en los dos puntos que quieres comparar. Es el modo más seguro.
3. **Corriente** (A): se mide **en serie**, es decir, **abriendo el circuito** y metiendo el multímetro como parte del cable. Es el modo peligroso: si lo usas mal, fundes el fusible del aparato (o algo peor). Aprende primero a medir voltaje.

Qué hace la industria

En un tablero industrial, el estándar de facto para sensores y controladores es **24 V de corriente continua**: es lo bastante alto para que el ruido eléctrico no confunda las señales a lo largo de cables largos, y lo bastante bajo para que sea menos peligroso que la corriente de la pared. Los robots móviles de almacén (los AGV y AMR, vehículos guiados y robots móviles autónomos) suelen usar baterías de 24 V o 48 V, y su paro de emergencia es **cableado**, no un botón de pantalla. El proyecto aplica el mismo principio a escala de laboratorio, y el capítulo 4 explica cómo.

Para ponerte a prueba

1. Un sensor de 24 V PNP se conecta por error a un pin de 3.3 V de la Raspberry. ¿Qué crees que pasa, y qué pieza intermedia debió ir en medio?

2. Si el ESP32 y el driver de motores no comparten GND, ¿por qué la señal "alto" que manda uno puede no ser entendida por el otro?
3. Con los datos de ejemplo del capítulo, ¿por qué una rampa cambia más la elección del motor que acelerar más rápido en piso plano?
4. Un fusible de 15 A en un circuito de 12 V: ¿cuántos vatios deja pasar como máximo antes de fundirse?
5. ¿Qué dato falta hoy para decidir el motor del robot y quién en tu equipo tiene que confirmarlo?

Capítulo 3: Estructura y mecánica del robot y del carrito

De qué está hecho el esqueleto, por qué un robot diferencial gira como gira, cómo funcionan la torre y el empujador, y qué dice la hoja de armado 3D (rev. 3.8) pieza por pieza.

Un plano que se puede girar con el mouse

Mario me pasó el archivo `Armado del carrito Aether.html`, una **hoja de armado interactiva en 3D**: se gira el modelo con el mouse, se avanza por etapas (unas cincuenta etapas de estructura, electrónica y funcionamiento, más una etapa por cada uno de los 107 cables de la placa), y cada etapa dice qué se hace, por qué, con qué herramientas, cómo comprobar que quedó bien y cuál es el error típico. Su versión de trabajo lleva fecha **16 de octubre** (tabla de medidas "rev. 3.4") con ajustes posteriores anotados hasta la **rev. 3.8** y la banda del **18 de octubre**.

Este capítulo y los siguientes de hardware se apoyan en ese archivo como **fuente del diseño**, y lo hacen con dos reglas que el propio archivo enseña a seguir:

1. **Todo número que el autor dibujó "a criterio" lleva un "~"** y aparece en una lista de *supuestos*. Aquí lo marco igual.
2. El archivo cita como fuentes `CARRITO_CONSTRUCCION_PASO_A_PASO.md`, `GUIA_ARMADO_PRINCIPIANTES.md`, `COSTOS_CARRITO.md` y `PROYECTO_INTEGRADOR_CONTEXTO.md`. **Esos documentos no están en el repositorio**, así que no puedo contrastar cada cifra con su origen.

Entonces: lo que sigue es **el diseño documentado por el equipo (rev. 3.8)**. No es una medición de un robot armado. Cuando digo "VERIFICADO" me refiero a que **lo leí en el archivo**, no a que alguien midió la pieza con un vernier. Si el carrito ya existe físicamente, los números de la hoja se confirman contra el real: **PENDIENTE (P-04)**, con Isaac (diseño y CAD) y David (ensamble).

Qué hace este carrito, en tres frases

La unidad móvil llega a la banda, **se acopla** a ella, y va tomando los *fixtures* (las bases que sostienen cada pieza; aquí cada fixture carga una **ECU**, la computadora de motor de un coche, de las marcas Siemens VDO y MaxxECU según el diseño de los fixtures) hasta apilar **6**. Luego viaja a la **estación de descarga** (una mesa inclinada) y, nivel por nivel, un

empujador los saca por una ventana. Es un robot de **transporte con elevador**, no un brazo: consecuencia directa de que el proyecto abandonó el KUKA y el UR5 a favor de banda + pistón (capítulo 1).

El esqueleto: aluminio 3030, escuadras impresas y tuercas T

La estructura es un marco de **perfil de aluminio 3030**: barras de 30 × 30 mm con ranura de 8 mm. Se une con **tuercas T de martillo**, tornillos Allen **M6** y **escuadras impresas en 3D**, sin soldar. Números del diseño:

Concepto	Valor (rev. 3.8)	Estado
Huella exterior	0.50 m (largo, hacia el frente) × 0.40 m (ancho)	VERIFICADO en la hoja
Postes verticales	4 barras de 0.95 m	VERIFICADO en la hoja
Altura total aproximada	≈ 1.04 m con rodajas (la guía original decía ≈ 1.03 m)	NO VERIFICADO (~)
Largueros / travesaños	4 de 0.44 m / 7 de 0.34 m (más uno de 198 mm para el empujador)	VERIFICADO en la hoja
Cortes de perfil	19 piezas, ≈ 11.4 m en total	VERIFICADO en la hoja
Piezas impresas	28 escuadras + 16 placas de unión, más soportes, cuna de batería, bandeja y guías	VERIFICADO en la hoja
Tuercas T + Allen M6	100 juegos	VERIFICADO en la hoja
Material de impresión de las escuadras	"40 % de relleno y 5 perímetros" (consejo de la hoja)	PENDIENTE (P-04): material y impresora no están dichos

Tres detalles que enseñan oficio y que debes copiar como hábito:

- **Las longitudes cuadran con aritmética.** 0.50 − 2 × 0.030 = 0.44; la hoja marca con una palomita cada cuenta comprobada.
- **El marco se escuadra con las diagonales.** En un rectángulo bien armado las dos diagonales miden igual: aquí $\sqrt{(0.5^2 + 0.4^2)} = 0.64$ m. La hoja pide igualarlas **a ±2 mm** *antes* de apretar tornillos, porque "todo lo de arriba hereda este marco".
- **Las tuercas T entran de frente.** Se meten de lado en la ranura y al apretar giran un cuarto de vuelta y se traban. Error típico que la hoja anota: apretar sin que hayan girado, y se salen.

Robot diferencial: girar sin volante

Tu robot es **diferencial**: **dos motores, una llanta por motor**, sin volante. Cómo se mueve (**conocimiento general**):

- Las dos llantas giran igual → avanza recto.
- La izquierda más rápido que la derecha → gira a la derecha, y al revés.
- Una avanza y la otra retrocede a igual velocidad → **gira sobre su propio eje**.

Con solo dos llantas el robot se caería hacia adelante o atrás, así que el diseño añade **4 rodajas** (ruedas locas) "twin" de 50 mm en las esquinas, que solo siguen la dirección que les marcan las llantas motrices. Cada rodaja mide **9 cm de alto con su espiga**: ese número fija la altura de todo el marco sobre el piso.

Las llantas motrices son **motor-rueda M0601** (rueda de **Ø 102 mm**, ancho ≈ 45 mm): el **izquierdo es el modelo FIT1042 y el derecho el FIT1038**, espejos uno del otro. La hoja avisa del error clásico: "*Montar dos del mismo lado: una rueda giraría al revés*". Cada uno trae **su propio driver y su propio encoder** (el capítulo 5 explica qué significa eso) y se maneja por un bus **RS485**.

Una contradicción que debes conocer. Mario me indicó el 24 de octubre que "*los motores aún no están elegidos*", mientras que esta hoja ya nombra el M0601 con sus modelos. Los dos datos están en el libro: la **hoja** es el diseño del equipo; lo que Mario dice es lo que él confirma hoy. **PENDIENTE (P-01):** confirmar si el M0601 es el motor definitivo.

Un problema de ingeniería real que la hoja resuelve con un número: con un diámetro de 102 mm la llanta sube hasta 10.2 cm, pero el marco queda a 9 cm del piso, **así que la llanta chocaría con el perfil central**. Por eso el eje de las ruedas se corre **7 cm hacia adelante** del travesaño central (el travesaño central se pone "por el costado de atrás": *por el frente pasan los motores de las ruedas*). Un error así se descubre dibujando; descubrirlo con el aluminio cortado cuesta dinero.

Suspensión: 85 newtons que mantienen la llanta pegada al piso

Cada llanta motriz va en una **placa pivotante**: dos baleros **608ZZ** a presión y un tornillo **M8** de pivote, con un **resorte** que la empuja al piso. La hoja da la receta exacta: resorte de **75 mm libres**, comprimido **28 mm** (a 47 mm), constante de unos **3 N/mm**, lo que da **≈ 85 N** de fuerza sobre cada rueda (85 N ≈ 8.7 kg-fuerza).

¿Para qué? Con dos llantas motrices y cuatro rodajas, una irregularidad del piso puede dejar una llanta **sin contacto**; sin contacto no hay tracción y el robot se desvía. El resorte garantiza una fuerza mínima. El balance, escrito por la hoja como error típico: "*Demasiada precarga levanta las rodajas y el carrito se mece; muy poca y las ruedas patinan al arrancar.*" Se ajusta **midiendo con vernier** en el robot real. Es el tipo de calibración que nunca sale igual en el mundo físico que en el papel.

Peso: tus números y los de la hoja

Mario dio estas **estimaciones de partida** del robot: **hasta 40 kg** en total y unos **17 kg sobre cada llanta motriz**, **PENDIENTE de confirmar (P-02)**. Haz la resta: 40 − 17 − 17 = **6 kg** para las rodajas: casi todo el peso va sobre las llantas que empujan, lo cual da agarre (capítulo 2) y obliga a que las rodajas no se levanten.

La **batería** del diseño es una **BR-1224 de 12 V, 24 Ah y 8.3 kg**, de ≈ 16.6 × 17.5 × 12.5 cm (medidas "típicas", marcadas ~), colocada **al centro y lo más abajo posible**, sujeta con topes impresos y una **correa con hebilla** alrededor del travesaño central. La razón, en palabras de la hoja: "*8.3 kg sueltos en un frenazo rompen algo. Abajo y al centro, baja el centro de gravedad.*"

(La versión anterior de la hoja que estaba en `dashboard/armado.html` hablaba de "~8.7 kg por rueda"; la rev. 3.8 ya no lo dice. Está anotado en `NOTAS_CORRECCIONES.md`.)

Centro de gravedad: el enemigo de las torres altas

El carrito mide poco más de 1 m y carga hasta 6 fixtures arriba. El **centro de gravedad** es el punto donde parece concentrarse todo el peso. Cuanto más alto está, más fácil es volcar al frenar, girar o tomar una pendiente. Regla general: un robot alto y angosto necesita su peso pesado abajo y centrado, y la **torre cargada** sube ese centro justo cuando el robot viaja a la estación de descarga. La altura real del centro de gravedad no existe como dato; calcularla con cifras inventadas es lo que el proyecto evita: **PENDIENTE (P-02)**.

La torre: un elevador que no se cae solo

El mecanismo que sube y baja la plataforma es un **elevador de husillo**:

- **Dos husillos TR8** de 60 cm (tornillos de potencia de 8 mm: al girar suben o bajan una tuerca) y **dos varillas lisas de 8 mm** con **6 baleros lineales SC8UU** que guían la plataforma.

- Lo mueven **dos motores a pasos NEMA 17 (modelo 17HS4401)** mediante **coples flexibles de 5 a 8 mm**, apoyados en **4 chumaceras KFL08**. Un **motor a pasos** avanza un "paso" fijo por cada pulso, así que "sabe" cuánto giró sin sensor. El husillo tiene **autobloqueo**: sostiene la carga sin gastar corriente (la hoja incluye una etapa "Finales de carrera y prueba de autobloqueo").
- **Tres finales de carrera** (interruptores de límite): dos del elevador (uno al fondo, 39.2 cm) y uno de *home* del empujador. Van como **contacto normalmente cerrado** para que un cable roto se lea como "presionado" y el motor se detenga (idea del capítulo 4).
- Recorrido útil: plataforma de **40.7 a 83.2 cm** del piso, con **6 niveles de 7.0 cm**: 0.407 + 6 × 0.070 = 0.827 m, que es justo la altura de la banda (83 cm) menos 3 mm.

La medida que más importa, y que la hoja revisa con palomitas: la **columna de la torre** tiene un hueco de **22.0 × 19.0 cm** y la **quilla** del fixture mide **21.3 × 16.5 cm**; las ECU sobresalen hasta ±11.15 cm del centro. La holgura entre fixture y guías es de **2 a 5 mm por lado**: lo mínimo para que entre sin atorarse ni bailar.

El empujador: cremallera, piñón y una uña con servo

Para **meter** el fixture a la pila y para **sacarlo** en la descarga, el carro lleva un **empujador**: una **cremallera** (barra dentada, 2 tramos de 15 cm = 30 cm) movida por un **piñón** (módulo 1.5, 20 dientes, Ø 30 mm) acoplado a un **tercer NEMA 17**. Recorre **25 cm** (su "home" es la posición 0.8 cm). Una **uña** (un dedo impreso de 7 mm que gira 90° sobre un eje, movido por un **microservo MG90S de 180°**; el de 360° no sirve) engancha la ranura del fixture para **jalarlo ~7.7 cm** hacia dentro desde la banda.

La carga en la banda: una cinta que no espera

Este es el truco de diseño más fino y está bien explicado en la hoja. La **cinta de la banda no se detiene**; el carrito no puede pedirle que pare. Entonces:

1. El carrito llega por **waypoints** (puntos de ruta), un "embudo" lo centra y la **cámara** del Jetson lee un **ArUco** para confirmar la posición.
2. Hace *home* del elevador y del empujador y deja la plataforma 3 mm **debajo** de la cinta (a 82.7 cm).
3. El fixture viaja sobre la cinta; cuando su centro pasa la **cresta del rodillo** (de Ø 50 mm en el extremo), su frente baja a la pila, la cinta lo sigue empujando hasta que la cola suelta la cresta y cae plano **≈ 6 cm afuera de la ventana**.

4. La uña baja, el empujador lo jala, sube la uña, regresa a home y la plataforma **baja un nivel (7 cm)**.
5. Se repite **uno a la vez** hasta 6. Un ciclo completo tarda ≈ 15 a 18 s; con la cinta a **0.20 m/s (supuesta)**, eso exige unos 3 m de separación entre fixtures. Por eso quien pone los fixtures espera el aviso "listo" del carrito.

Honestidad de la propia hoja: la velocidad real de la cinta y el punto exacto donde cae el fixture **se miden con la banda real**; el valor "6 cm" depende de que la cinta agarre menos que el PETG sobre PETG (supuesto).

La banda (de Isaac) y la mesa de descarga

La banda sale del archivo **STEP de SolidWorks de Isaac**: su modelo es un **tercio** de la banda real, así que el diseño pega **3 tramos de 1030 mm (3.09 m)** con **una sola cinta de 20.35 cm** de ancho. Las patas no vienen en el STEP y se dibujan con perfil 3030. Datos que importan:

- **Altura de la cinta: 83.0 cm** (supuesta; en el STEP la cinta queda **8.5 mm debajo del borde de los rieles**, así que si los rieles están a 83 cm la cinta está a 82.15 cm: "hay que subir la banda o bajar 8.5 mm la altura de carga").
- Guías laterales de 3 mm a 2 mm de la quilla, colgadas de los rieles cada ~0.5 m (**propuesta**: la banda real no trae guías y entre rieles hay 21.6 cm).
- **Mesa de descarga:** una **mesa inclinada de melamina de 1.4 × 0.3 m a ≈ 15°**, con bisagra, cuña, guías y tope de hule espuma (dibujados a criterio). Al empujar el fixture por la ventana, cuando su centro pasa la orilla de la mesa **se vuelca y resbala solo**; en una mesa plana el empujador tendría que arrastrar toda la fila y no le alcanzaría la fuerza (explicación de la hoja).

La cara de enfrente: ventana de MDF

El frente del carrito lleva un **panel de MDF de 3 mm** con una **ventana de 24 × 9.7 cm** (borde inferior a 81.8 cm, 1.2 cm bajo la banda). El panel es además el **tope** de la pila. Detalle de la hoja: las columnas delanteras están separadas **23.2 cm** (a z ±0.131) para que pasen las ECU, mientras que las traseras se quedan más juntas.

Qué dice la hoja y qué dices tú

Tema	Hoja de armado (rev. 3.8)	Lo que dice Mario (24 oct)	Estado
Motores	2 × M0601 (FIT1042/FIT1038), 18 V, RS485	"Aún no están elegidos"	PENDIENTE (P-01)
Peso total	No lo da (batería 8.3 kg)	Máximo estimado 40 kg	PENDIENTE (P-02)
Peso por llanta	No lo da (la versión vieja decía ~8.7 kg)	~17 kg por llanta motriz	PENDIENTE (P-02)
Cómputo	Jetson Orin Nano Super, "ya comprado"	Jetson Orin Nano Super corriendo el agente	VERIFICADO (bitácora)
Cámara	Logitech 1080p, "ya la tiene Mario"	Cámara USB UVC reportada en el Jetson	VERIFICADO (bitácora)
Microcontrolador	ESP32 DevKitC de 38 pines	"Ya tengo el ESP32"	VERIFICADO que lo tienes; PENDIENTE (P-08) el modelo exacto
Armado físico	Hoja con unas cincuenta etapas más 107 pasos de cableado	—	PENDIENTE (P-04): ¿ya hay estructura armada?

Cómo se prueba la mecánica antes de energizar

La hoja termina cada etapa con una **comprobación física**. Cuatro que cualquier mecatrónico entiende:

1. "*Las 2 ruedas y las 4 rodajas tocan el piso al mismo tiempo.*" Si una no toca, el robot cojea.
2. "*Diagonales del marco de abajo iguales (±2 mm).*" Sin eso, el robot no anda derecho.
3. "*La batería no se mueve aunque sacudan la base; empujada a mano, la base rueda derecho unos 3 m.*"
4. "*Resortes comprimidos 28 mm, medidos con vernier.*"

Nota honesta: **no hay evidencia en el repositorio de que el carrito ya esté armado.** Las pruebas de arriba son las que la hoja prescribe, no las que alguien reportó haber hecho. Quién las hace y cuándo es de David e Isaac: **PENDIENTE (P-04)**.

Qué hace la industria

Los robots de almacén que mueven estantes o cajas (**AGV**, vehículos guiados automáticamente, y **AMR**, robots móviles autónomos) usan casi el mismo esquema: tracción diferencial, ruedas locas de apoyo, batería baja para bajar el centro de gravedad y un mecanismo elevador. Lo que cambia es la certificación: los industriales llevan sensores de seguridad homologados y paros de emergencia certificados. El capítulo 27 revisa casos reales.

Para ponerte a prueba

1. Una llanta izquierda a 0.3 m/s y la derecha a 0.1 m/s: ¿hacia dónde gira el robot?
2. ¿Por qué la hoja mueve el eje de las ruedas 7 cm hacia adelante del travesaño central?
3. Con 40 kg y 17 kg por llanta motriz, ¿cuántos kilos quedan para las rodajas, y por qué eso es bueno y malo a la vez?
4. ¿Qué compruebas con las dos diagonales de un marco rectangular y a qué tolerancia?
5. ¿Por qué la cinta no se detiene durante la carga, y qué consecuencia tiene para quien coloca los fixtures?
6. La hoja y tú no coinciden en los motores: ¿a quién preguntas y qué haces con el capítulo mientras tanto?

Capítulo 4: Alimentación, baterías, protecciones y paro de emergencia

De dónde sale la energía del carrito, por qué se reparte en ramas, qué hace cada fusible, convertidor y relé, y por qué el botón rojo no es software.

Un diagrama de bandeja que se lee como un cuento

Si abres la hoja de armado y llegas a la **Etapa 11**, el armado de la electrónica de potencia se cuenta en el orden en que hay que hacerlo: primero **se ajustan los voltajes sin conectar nada**, después fusibles y relé, y al final una **prueba de humo** con la batería pero **sin ESP32 ni motores**. Ese orden es la lección entera del capítulo: *la electricidad se prueba por capas, de lo que menos cuesta quemar a lo que más.*

Una nota sobre fuentes. La guía de construcción del 9 de octubre describe un robot con **Raspberry Pi 5** y driver TB6612. Tú me aclaraste que la Pi 5 fue un plan del 30 de septiembre que se descartó y que, ante contradicción, **gana la bitácora del Jetson**. La arquitectura eléctrica que sigue sale de la **hoja de armado rev. 3.8**, que ya usa el **Jetson Orin Nano Super**. Es el **diseño del equipo**, no la medición de un carrito encendido: los datos finales de batería, convertidores y cableado los confirma David: **PENDIENTE (P-05, P-21)**.

La idea de fondo: islas de voltaje y un solo cero

En el carrito conviven varios voltajes, y cada uno tiene su propia isla:

Voltaje	De dónde sale	Qué alimenta
12 V (batería)	Batería de plomo de 12 V y 24 Ah, "BR-1224", de 8.3 kg	Todo lo demás, repartido en dos ramas
18 V	**Elevador de voltaje de 400 W** (12 → 18 V)	Los dos motores-rueda M0601
12 V "de pasos"	La rama principal, ya pasada por el relé	Los 3 drivers **A4988** de los motores a pasos
5 V	**Reductor XL4005** (12 → 5 V), ajustado a 5.0–5.1 V	ESP32, módulo MAX485, servo de la uña y tira de LEDs
3.3 V	Regulador del propio ESP32	Lógica de los A4988, sensores VL53L0X, BNO055, PCF8574 y barra QTR-8A

Y **un solo cero**: el capítulo 2 explicó la tierra común. En este diseño todas las tierras se unen **en el borne IN− del elevador de voltaje**, a donde llega el negativo de la batería: "*Todas las tierras unidas: batería −, elevador, XL4005, drivers y ESP32*" (palabras de la hoja). Un solo punto evita que las corrientes grandes de los motores viajen por cables delgados de señal.

Las dos ramas de la batería

Del borne de la batería sale un cable rojo de **14 AWG** (calibre americano: 14 es grueso, para corrientes grandes) a un conector **XT60** (el enchufe amarillo de baterías; hembra del lado de la batería, macho del lado del carrito). Enseguida hay **dos portafusibles**, y ese es el corazón de la protección:

Rama principal: fusible de 15 A → relé → motores. El fusible alimenta el contacto **COM** de un **módulo de relé T90 de 30 A** de 2 canales. Del contacto **NO** ("normalmente abierto": solo cierra cuando la bobina recibe energía) salen dos ramas: la entrada del elevador de 18 V (para las ruedas) y la bornera **J1** de la placa (12 V de los drivers de los motores a pasos). El relé es el interruptor general de la **potencia**.

Rama auxiliar: fusible de 10 A → interruptor KCD1 → lógica. El otro fusible pasa por un **interruptor KCD1** (el de encendido del robot) y llega al **XL4005**, que reparte: 5 V al ESP32 y accesorios; **12 V** al **Jetson** (cable terminado en un plug de 5.5 × 2.5 mm, centro positivo); y 12 V a la **bobina** del relé pasando por el paro de emergencia. Una consecuencia: **apagar el KCD1 también suelta el relé**, porque le quita la energía a la bobina: con el robot "apagado" no queda potencia en las ruedas.

¿Por qué dos fusibles? Porque cada rama tiene un temperamento distinto:

- La de **motores** es potente y brusca: fusible grande para que los arranques normales no lo fundan.
- La **auxiliar** es delicada y sensible a caídas de voltaje: fusible más chico que se funde ante un corto antes de que se caliente un cable fino.
- Si se funde el de motores, el robot se detiene **pero el Jetson sigue vivo y sabe qué pasó**. Con un solo fusible perderías las dos cosas a la vez.

Regla de la hoja sobre dónde ponerlo: *"El fusible va lo más cerca posible de la batería: si algo hace corto, se quema el fusible y no el cable."*

Qué significa "BR-1224" y por qué la autonomía se mide, no se calcula

La etiqueta de una batería dice tres cosas:

- **12 V**: voltaje nominal.
- **24 Ah** (amperios-hora): capacidad. En teoría, 24 A durante 1 hora o 1 A durante 24 horas.
- **Energía** = 12 × 24 = **288 Wh** (vatios-hora).

Si solo estuviera conectado el Jetson trabajando a 25 W (el modo en que hoy corre, **VERIFICADO** en la bitácora), 288 Wh darían unas **11 horas** en el mejor caso teórico (**cuenta de ejemplo**, no medida). Motores, pérdidas de los convertidores y la regla general de no descargar una batería de plomo a fondo recortan eso mucho. **La autonomía real se mide con el robot completo trabajando.** El modelo exacto de batería sigue siendo **PENDIENTE (P-05)**.

Los voltajes se ajustan antes de conectar nada

La hoja lo dice con claridad: "*La mayoría de las piezas se queman por un voltaje equivocado. Marquen cada módulo con su voltaje en cinta.*" Receta de la Etapa 11:

1. Con el módulo conectado a 12 V y **sin nada en la salida**, gira su potenciómetro (un tornillito) con un desarmador de plástico hasta leer con el multímetro **5.0–5.1 V** en el XL4005 y **18.0 V** en el elevador.
2. Ajusta el **Vref** de cada driver A4988 **sin motor conectado**: **0.80 V** (si la resistencia de sentido es R100) o **0.54 V** (si es R068), lo que limita cada motor a **1.0 A**. El **Vref** es un voltaje de referencia dentro del driver que fija cuánta corriente máxima deja pasar al motor: demasiado alto y el motor se calienta; demasiado bajo y se salta pasos.
3. Anota los valores (la hoja deja renglones para eso).

Dos reglas de seguridad que la hoja repite: **nunca conectes ni desconectes un motor a pasos con el driver encendido** (quema el A4988) y **mide antes de enchufar el plug del Jetson** que el centro sea el positivo (la polaridad al revés lo daña).

La prueba de humo: probar sin arriesgar

Antes de montar módulos delicados, la hoja pide una **prueba de humo** (se llama así porque, si algo está mal, sale humo): conectar la batería con el XT60 y el KCD1 encendido, **sin ESP32 y sin motores**. Se mide **5 V** en la placa y **12 V** antes del relé; el relé hace *clic*; se aprieta el paro y debe soltar, dejando el contacto NO en **0 V**. Se esperan 2 minutos y se tocan los módulos: "*Ninguna pieza caliente ni con olor.*" Si algo falla, quemas un fusible o un regulador de unos pocos pesos, no el ESP32 ni los drivers. Antes de todo eso, con el XT60 **desconectado**, el multímetro en continuidad entre + y − **no debe pitar** (no hay corto).

El paro de emergencia: cómo está pensado en este diseño

Aether ya tiene en el código un botón "Paro" en la pantalla de la celda. Léelo con atención: el propio archivo `backend/cell_hmi.py` aclara en su comentario inicial que "*El paro de emergencia no pasa por aquí*". Es deliberado: **un botón de pantalla es software**, y el software puede estar colgado, sin red o mal escrito justo cuando más lo necesitas. Un **paro de emergencia** real es un **interruptor físico que corta la energía de lo peligroso, sin depender de que nada funcione**.

El diseño del carrito lo resuelve así:

1. **Botón LAY37 con contacto normalmente cerrado (NC).** Mientras está suelto deja pasar corriente; al pulsarlo, **abre** el circuito. Si un cable se rompe, también se abre: **falla hacia lo seguro**. Un contacto normalmente abierto haría lo contrario y un cable roto dejaría el paro sin efecto sin avisar.
2. **El paro no corta la potencia directamente: alimenta la bobina del relé.** El cable `12 V aux → contacto NC del paro → VCC del relé` hace que el relé **solo esté cerrado mientras el paro esté suelto**. Al pulsarlo, la bobina se queda sin energía, el relé **cae solo** y corta **a la vez** el elevador de 18 V (ruedas) y los 12 V de los drivers de los motores a pasos (elevador de plataforma y empujador). Frase de la hoja: "*aunque el programa esté colgado*".
3. **La rama auxiliar sigue viva**: Jetson y lógica **no se apagan** al pulsar el paro. El robot se detiene pero sigue "pensando" y puede reportar lo ocurrido.
4. **Dónde queda:** "*arriba, en una esquina de atrás*" del carrito, **al alcance de la mano** en cualquier momento del ciclo. Un botón al que hay que rodear el robot para llegar no es un paro.
5. **Prueba que pide la hoja:** "*El paro detiene ruedas, elevador y empujador al instante y la plataforma se queda donde estaba.*"

Un hueco de diseño que conviene ver

Leí el cableado de la placa completo (107 cables). **El paro solo va al relé: no hay cable del paro hacia un pin del ESP32.** Eso significa que, en este diseño, **el software no se entera de que se pulsó el paro**: el ESP32 podría seguir mandando órdenes de movimiento a motores sin energía. No es peligroso mientras el paro está pulsado, pero **al soltarlo los motores recuperan potencia y podrían moverse de golpe** si el programa conserva la última orden. Es una decisión de diseño de David y Mario: *¿se agrega una entrada para que el software detecte el paro y frene antes de soltarlo?* **PENDIENTE (P-06).** (La guía vieja del 9 de octubre sí tenía ese segundo circuito hacia un pin; la hoja nueva no lo trae.)

El perro guardián: la segunda red

La hoja dice, del cable USB entre el Jetson y el ESP32: "*Si se desconecta, el watchdog detiene las ruedas en 1 s.*" Un **watchdog** ("perro guardián") es un temporizador que obliga a detener el robot si el programa principal deja de mandar su "latido" a tiempo: protege contra que el cerebro se cuelgue. Ojo: **el programa del ESP32 para este diseño todavía no existe en el repositorio** (`firmware/` solo contiene archivos `.gitkeep`, vacíos). El watchdog es un diseño, no una función que ya corra. Lo vemos en el capítulo 5.

Cómo se mide la batería sin freír un pin

Para que el robot sepa cuánta batería le queda, la hoja usa un **divisor de voltaje**: una resistencia de **100 kΩ** y una de **20 kΩ** en serie, con un capacitor de **100 nF** para limpiar el ruido. Con 14.4 V (batería llena) el punto medio tiene 14.4 × 20 / (100 + 20) = **2.4 V**, un valor seguro para un pin de 3.3 V. Ese punto va al **GPIO 39** del ESP32 (solo entrada analógica). La fórmula de la hoja: **voltaje real ≈ lectura / 4095 × 3.3 × 6**. Regla general que ilustra: **nunca conectes 12 V directo a un pin de 3.3 V**; usa un divisor.

Un detalle para revisar con David: dos caminos de 5 V hacia el ESP32

La hoja alimenta el ESP32 por su pin de **5 V desde el XL4005**, "*no por la computadora*"; y a la vez lo conecta por **micro USB al Jetson** para datos. Regla general: cuando una placa recibe 5 V por dos caminos a la vez, las fuentes pueden "pelearse" salvo que haya un diodo o que el cable USB no lleve la línea de alimentación. Cómo se evita aquí no está documentado: **PENDIENTE (P-21)**.

Cables: por qué el grueso importa

La hoja especifica calibres: **14 AWG** para la batería y la rama de motores, **16–18 AWG** hacia los motores-rueda, **18–20 AWG** para drivers y lógica, **22 AWG** para señal. Error que anota: "*Cable de potencia delgado (22 AWG) en la rama de 15 A: se calienta y se derrite.*" Y otra regla: **potencia por un lado y señal por otro**, con un cincho cada ~10 cm, porque "*Cables de señal pegados a los de potencia: los VL53L0X y el RS485 fallan 'al azar'.*"

La celda: 24 V y 5 V separados

La **celda de manufactura** tiene su propio mundo eléctrico, según la hoja de conexión y la guía:

- Una **fuente de 24 V del tablero** alimenta el controlador, los dos infrarrojos y la bobina del relé del pistón.
- La **Raspberry Pi 4** se alimenta con **su propia fuente de 5 V**, aparte. "*No hay hilo de 24 V hacia la Pi.*"
- La Pi **no energiza** el pistón: comanda un **relevador** que sí lo hace (capítulo 7).

El mismo principio: **cada voltaje en su isla, y entre islas se comunica por relé o por red, nunca por cobre directo.** El **paro de emergencia físico de la celda** no aparece en ningún documento del repositorio: **PENDIENTE (P-17)**, con David y Pedro.

Los errores que queman cosas (y que tu repo ya escribió)

Recopilados de la hoja y de la guía:

- Meter 24 V a un pin de la Pi: "*24 V lo quema.*"
- Conectar un motor directo a un pin: un pin entrega unos pocos miliamperios; un motor puede pedir varios amperios.
- Conectar o desconectar un motor a pasos con el driver encendido: quema el A4988.
- Olvidar el capacitor de **100 µF** junto a cada A4988, que amortigua los picos. Con la polaridad al revés "*se infla o revienta*".
- Poner **VDD del driver a 5 V** en vez de 3.3 V: no entiende las señales de 3.3 V del ESP32.
- Alimentar la barra QTR-8A a **5 V** cuando sus salidas van a pines de 3.3 V: "*mandaría 5 V a pines que aguantan 3.6 V.*"
- Alimentar el servo desde el ESP32 en lugar del XL4005: "*al moverse jala corriente y reinicia la placa.*"

- Conectar la batería **al revés** (+ con −): "*quema el ESP32 o el módulo al instante.*"
- Montar un módulo **al revés** en su zócalo.
- Dejar la placa tocando el aluminio: "*un corto en cuanto el carrito vibra.*" (Por eso la bandeja va con separadores M3.)

Norma y estándares: qué existe, sin fingir que lo cumples

Para máquinas, el paro de emergencia está regulado por normas internacionales: en general **IEC 60204-1** (equipo eléctrico de máquinas) e **ISO 13850** (función de paro de emergencia) (**conocimiento general**, no verificado en tu proyecto). Aether es un **prototipo de laboratorio** y no está certificado; lo honesto ante el jurado es decir: "*seguimos los principios (contacto NC, corte de potencia independiente del software), no tenemos certificación.*" El propio `cell_hmi.py` usa la misma humildad: su cuenta de pantalla "*no es una certificación del sistema*".

Qué hace la industria

En un robot de almacén comercial el paro de emergencia está **homologado**, hay botones en varios lados, la batería va en una caja con protección contra corto y sobretemperatura, y la potencia (24 V o 48 V) se corta con un **contactor de seguridad** que no depende del cerebro. La lógica es la misma que la de tu relé controlado por el paro, a otra escala y con papeles firmados.

Para ponerte a prueba

1. ¿Por qué el contacto del paro es "normalmente cerrado" y no "normalmente abierto"?
2. Si el Jetson se cuelga con las ruedas a toda velocidad, ¿qué corta la potencia primero: el programa o el relé del paro? Y si pulsas el paro, ¿el Jetson se apaga?
3. Una batería de 12 V y 24 Ah: ¿cuántos vatios-hora guarda y por qué la autonomía real será menor?
4. ¿Por qué hay un fusible de 15 A y otro de 10 A, y por qué el de 10 A pasa por un interruptor?
5. En un divisor de 100 kΩ y 20 kΩ, ¿qué voltaje llega al pin con 12.6 V de batería?
6. ¿Qué podría pasar al soltar el paro si el ESP32 no se enteró de que fue pulsado, y qué decisión de diseño queda abierta?

Capítulo 5: Motores, drivers y el ESP32

Cómo se mueven las ruedas, el elevador y el empujador; qué es un driver, un motor a pasos y un bus RS485; y qué pin del ESP32 hace qué en el diseño de tu carrito.

El cerebro piensa lento y el músculo reacciona rápido

Aether tiene dos computadoras en el robot con trabajos opuestos. El **Jetson** piensa: ve la cámara, reconoce piezas, decide a dónde ir. Pero Linux, el sistema que corre en el Jetson, no está hecho para responder a un evento en microsegundos: puede "distraerse" un instante con cualquier otro proceso. Mover motores exige lo contrario: pulsos con la separación exacta, lectura de contadores, frenar al instante.

Para eso existe el **ESP32**, un **microcontrolador**: un chip que ejecuta **un solo programa**, sin sistema operativo que lo interrumpa, con pines que puede encender, apagar o leer con precisión. Es el "cerebelo" del robot: recibe del Jetson órdenes de alto nivel ("avanza", "sube el elevador") y las convierte en pulsos y señales. Tú me dijiste que **ya tienes el ESP32 en la mano** y que **todavía no has decidido con qué software lo programas**; el diseño de la hoja de armado usa un **ESP32 DevKitC de 38 pines** (tarjeta de desarrollo con chip ESP32, regulador de 3.3 V y puerto micro USB). El modelo exacto de tu placa: **PENDIENTE (P-08)**.

Tres tipos de motor en un solo robot

El diseño usa tres clases de actuadores y conviene no confundirlos:

Actuador	Cuántos	Para qué	Cómo se manda
Motor-rueda M0601 (modelos FIT1042 y FIT1038)	2	Mover el carrito	Por un bus **RS485**: el ESP32 le manda órdenes en mensajes y el motor trae su propio driver y encoder
Motor a pasos NEMA 17 (17HS4401)	3	2 husillos del elevador + 1 piñón del empujador	Pulsos **STEP** y **DIR** a un driver **A4988**
Servo MG90S (180°)	1	La uña del empujador	Una señal **PWM** a un pin

(La existencia del M0601 en el diseño choca con tu comentario de que "los motores no están elegidos": **PENDIENTE (P-01)**, capítulo 3.)

Motor de corriente continua, PWM y driver: la llave y la tubería

Un **motor de corriente continua** (DC) gira más rápido cuanto más voltaje recibe. Pero un pin del ESP32 solo da 3.3 V y unos pocos miliamperios: no puede alimentar un motor. La solución es un **driver**: un circuito de potencia que **obedece** a una señal chica y **entrega** la corriente grande desde la batería. Es como la llave de la manguera: la mano (la señal) es débil, pero controla un chorro enorme.

¿Y cómo se controla la velocidad si solo hay "encendido" o "apagado"? Con **PWM** (*Pulse-Width Modulation*, modulación por ancho de pulso): se enciende y apaga el motor miles de veces por segundo y se cambia la **proporción de tiempo encendido**. 25 % encendido ≈ el motor gira a un cuarto de potencia. Es como pulsar el acelerador de un auto muy rápido: el motor "promedia".

El interior de un driver de motor DC es un **puente H**: cuatro interruptores en forma de H que, combinándose, dejan pasar la corriente en un sentido o en el contrario (adelante o atrás). La **guía vieja del 9 de octubre** usaba para esto un **TB6612FNG**, chip pequeño de ≈ 1 A por motor; la propia guía avisaba que "*si el robot es de tamaño persona, este driver no se usa*". Con 40 kg, no es el driver. En el diseño actual **las ruedas traen su propio driver dentro del motor**, así que el ESP32 no maneja corriente de ruedas, solo manda mensajes.

Las ruedas: órdenes por un bus RS485

El **RS485** es un estándar para comunicar equipos industriales con **dos cables trenzados** (**A** y **B**) a distancias largas y con mucho ruido eléctrico. Varios dispositivos cuelgan del mismo par de cables ("bus"), cada uno con su **ID**: aquí los dos M0601 son el **ID 1 y el ID 2**.

El ESP32 habla en 3.3 V, pero el chip **MAX485** que traduce a RS485 trabaja a 5 V, así que el diseño añade un **convertidor de nivel** (un traductor 3.3 ↔ 5 V). El cableado exacto, según la hoja:

Señal	Pin del ESP32	Qué hace
DI del MAX485	GPIO 17	Lo que el ESP32 **manda** a los motores
DE + RE (unidos)	GPIO 4	En 1, el MAX485 **habla**; en 0, **escucha**. El ESP32 lo cambia en cada mensaje
RO del MAX485	GPIO 16 (pasando por el convertidor de nivel)	Lo que **contestan** los motores, sale a 5 V y se baja a 3.3 V antes de entrar

Es un bus **half-duplex**: no se puede hablar y escuchar a la vez, por eso DE y RE van puenteados con un solo pin. Dos reglas de la hoja: **trenzar A con B** y **si los motores no contestan, invertir A y B** (síntoma típico de un RS485 recién armado).

Lo que no sabemos: el **protocolo** de mensajes del M0601 (qué bytes significan "gira a tal velocidad") **no está en el repositorio**. Hay que leer la hoja del fabricante: **PENDIENTE (P-09)**. Sin eso no se puede escribir el programa de las ruedas.

Odometría: saber cuánto avanzó el robot

La hoja dice que *"el ESP32 cuenta las vueltas de los M0601 a 50 Hz para la odometría"*. La **odometría** es estimar la posición del robot sumando cuánto giró cada rueda, como el cuentakilómetros de un auto. Una **vuelta** de rueda de Ø 102 mm avanza π × 0.102 ≈ **0.32 m**. Con las dos ruedas separadas **0.285 m** (±0.1425 m del centro, según la tabla de medidas), la cinemática diferencial (**conocimiento general**) es:

- Velocidad hacia adelante del robot: **v = (v_der + v_izq) / 2**
- Giro: **ω = (v_der − v_izq) / 0.285**

El viaje, según la hoja, se limita a **0.35 m/s y 0.4 m/s²**. La odometría **se acumula con error** (las ruedas patinan un poco), por eso el diseño la corrige con el **ArUco** que el Jetson ve con la cámara al llegar a cada estación (capítulo 14) y con una **IMU** (ver capítulo 6). Los **encoders** (sensores que cuentan cuánto giró el eje) vienen dentro de los M0601.

Motores a pasos: el motor que sabe contar

Un **motor a pasos** (NEMA 17 es solo su tamaño de brida: 42 × 42 mm) avanza un ángulo fijo por cada pulso. El 17HS4401 da **200 pasos por vuelta**. Para que gire más suave y silencioso, el driver **A4988** puede subdividir cada paso en **micropasos**; el diseño pone los pines MS1, MS2 y MS3 a 3.3 V, que activan el **micropaso 1/16**: **200 × 16 = 3200 pulsos por vuelta**. La hoja lo resume: "*Por eso 3200 pasos = 1 vuelta.*"

El A4988 recibe tres señales por motor:

- **STEP**: cada pulso es un micropaso.
- **DIR**: el nivel (alto o bajo) decide el sentido.
- **ENABLE**: en 0 el motor tiene fuerza (retiene); en 1 queda suelto.

Cuentas con las piezas de tu carrito

Con la hoja, puedes calcular el tamaño de cada movimiento (**ejemplos de cuenta, usando el micropaso 1/16**):

- **Empujador:** piñón de módulo 1.5 y 20 dientes, Ø30 mm → avanza π × 30 ≈ **94.2 mm por vuelta**. Una carrera de **250 mm** son 250 / 94.2 ≈ 2.65 vueltas ≈ **8 490 pulsos**. La hoja dice que la cremallera se mueve a **80 mm/s**: eso son 80 / 94.2 × 3200 ≈ **2 700 pulsos por segundo**, y la carrera completa dura ≈ 3 s.
- **Elevador:** un nivel son **70 mm**. Con un husillo de paso **P** mm (lo que avanza por vuelta), son 70 / P vueltas y 70 / P × 3200 pulsos. Si P = 8 mm: **28 000 pulsos**; si P = 2 mm: **112 000 pulsos**. Mira qué enorme es la diferencia: por eso la hoja dice "*medir el avance al recibirlos*". **El paso real del husillo TR8 es PENDIENTE (P-10)** y cambia todo el programa del elevador.

Los **dos motores del elevador** comparten STEP, DIR y ENABLE: un mismo pulso llega a ambos drivers, así que giran **exactamente igual**. Eso funciona mientras ninguno pierda pasos; no hay un sensor que compare los dos. La única corrección son los **finales de carrera**.

El mapa de pines del ESP32 (diseño rev. 3.8)

Este es el "guion" que el programa del ESP32 tendrá que respetar. Lo leí del cableado de la placa (cable por cable). **No está probado en hardware**: es diseño.

Pin (GPIO)	Función	Va a
25	STEP del elevador	A4988 #1 y #2
26	DIR del elevador	A4988 #1 y #2
27	STEP del empujador	A4988 #3
14	DIR del empujador	A4988 #3
13	ENABLE (de los 3 drivers)	A4988 #1, #2 y #3, con resistencia de 10 kΩ a 3.3 V
17 / 4 / 16	RS485: DI / DE+RE / RO	MAX485 (ver arriba)
21 / 22	I^2C: SDA / SCL	VL53L0X, PCF8574, BNO055 (capítulo 6)
18 / 19 / 23	Finales de carrera 1, 2 y 3	Elevador izquierdo, elevador derecho, home del empujador
32, 33, 34, 35, 36	Salidas S3 a S7 de la barra QTR-8A	Barra de línea (capítulo 6)
39	Voltaje de la batería	Divisor 100 kΩ / 20 kΩ (capítulo 4)

Pin (GPIO)	Función	Va a
5	PWM del servo	MG90S de la uña
2	Datos de la tira WS2812 (opcional)	Resistencia de 330 Ω a la tira de LEDs

Detalles de ingeniería que esa tabla esconde, y que son buena escuela:

- **La resistencia de 10 kΩ en ENABLE.** Al encender el ESP32, sus pines tardan unos instantes en "tomar posición". Con ENABLE atado a 3.3 V, los motores **quedan sueltos** durante el arranque; así no hay movimientos raros al encender. Palabras de la hoja: "*no hay movimientos raros al encender*".
- **Finales de carrera NC con pull-up.** Si el cable se rompe, el pin lee igual que "presionado" y el motor se detiene en lugar de chocar. Es la misma idea de falla segura del paro de emergencia.
- **Pines solo de entrada.** En el ESP32, los GPIO 34, 35, 36 y 39 **solo pueden leer** (no escribir), por eso se asignan a la barra y a la batería. Es **conocimiento general** del ESP32.
- **Pines con trampa al arrancar.** Algunos pines del ESP32 (como el 2 y el 5) afectan cómo arranca el chip; la hoja nota que los LEDs van sin convertidor de nivel "*si no, el ESP32 no se deja programar*". Por eso conviene dejar esas líneas desconectadas al subir un programa. Es una **alerta general**, no medida.

El perro guardián y el contrato con el Jetson

Entre el Jetson y el ESP32 va un **cable USB de datos** ("los de solo carga no sirven", advierte la hoja). Por ahí viaja el **contrato**: qué mensajes se mandan y cómo. Ese contrato **no está escrito** para este hardware. Tienes de ejemplo el que traía la guía vieja (del diseño con el TB6612): líneas de texto terminadas en salto de línea, `H` como **latido**, `M <izq> <der>` para velocidades de −100 a 100 y `S` para frenar; el ESP32 frenaba solo si pasaban **300 ms sin latido**. Es una **buena plantilla**, pero con otros motores, otro bus y otra lógica, habrá que escribir el contrato nuevo. La hoja pide un watchdog de **1 s**. Queda por escribir ese contrato nuevo (**PENDIENTE (P-25)**): qué mensajes existen para ruedas, elevador, empujador, uña y lecturas; cada cuánto manda el Jetson el latido y a los cuántos milisegundos frena el ESP32; y qué hace el ESP32 cuando se pulsa el paro (capítulo 4, **P-06**).

¿Con qué programo el ESP32?

Esa pregunta quedó abierta contigo. Estas son las opciones **reales** (conocimiento general) y mi recomendación, que es solo una opinión para que decidas:

Opción	Qué es	Pros	Contras
Arduino IDE	Editor sencillo con las librerías de Arduino sobre el ESP32	El camino más corto a "ya parpadea", montones de ejemplos	Editor básico, sin pruebas automáticas
PlatformIO (dentro de VS Code)	Extensión que usa las mismas librerías de Arduino con mejor proyecto	Se integra con tu Git y tu editor, fija las versiones de las librerías	Un poco más de configuración inicial
ESP-IDF	El kit oficial del fabricante, en C	Máximo control y rendimiento	Curva de aprendizaje alta para un solo programador con tiempo limitado
MicroPython	Python dentro del chip	Familiar para ti	Más lento; temporización crítica de pulsos es más difícil

La guía vieja ya traía un programa **para Arduino IDE** con la placa "ESP32 Dev Module" y la advertencia de que la función de PWM `ledcAttach` pide el **núcleo de ESP32 versión 3.x**. **Recomendación (opinión):** PlatformIO con el framework Arduino, porque ya hay librerías para **A4988**, **VL53L0X**, **BNO055** y **PCF8574** en ese ecosistema y te deja el proyecto dentro del mismo repositorio. La decisión es tuya: **PENDIENTE (P-07)**.

Cómo probar sin riesgo (reglas de oro)

La guía vieja lo dejó bien dicho y vale hoy: *"Las ruedas no se prueban en el suelo hasta que S y el botón de paro las hayan detenido en el aire."* Secuencia segura que se desprende de la hoja y la guía:

1. **Con el robot levantado**, ruedas sin tocar el piso.
2. Prueba de humo de la electrónica de potencia (capítulo 4).
3. Antes de conectar motores a pasos, ajusta el **Vref** de cada A4988 **sin motor**.
4. Mueve el empujador **a mano** su recorrido de 25 cm y revisa que no roce cables.
5. Primero lógica (USB), después potencia, después movimiento. Un cambio a la vez.

Qué hace la industria

En robots comerciales, el nivel bajo (motores, encoders, seguridad) lo ejecuta un **controlador de movimiento** dedicado, y el nivel alto (percepción, planificación) corre en una computadora distinta; se comunican por **CAN bus**, **EtherCAT** u **RS485/Modbus**. Aether usa la misma idea con piezas de laboratorio: ESP32 como controlador de movimiento y RS485 para las ruedas. Y la **seguridad** (paro) siempre se cablea aparte del software.

Para ponerte a prueba

1. ¿Por qué un ESP32 maneja mejor los pulsos de un motor a pasos que el Jetson?
2. ¿Qué diferencia hay entre STEP, DIR y ENABLE en un A4988?
3. Con 3200 micropasos por vuelta y un piñón que avanza 94.2 mm por vuelta, ¿cuántos pulsos necesita el empujador para recorrer 250 mm?
4. ¿Por qué DE y RE van unidos en el MAX485 y qué significa "half-duplex"?
5. ¿Para qué sirve la resistencia de 10 kΩ en el pin ENABLE al encender el robot?
6. Si el paso real del husillo resulta ser de 2 mm y no de 8 mm, ¿qué parte del programa del elevador cambia y por cuánto?

Capítulo 6: Sensores y cámaras

Cómo "siente" Aether: una cámara que lee códigos, sensores de distancia que frenan antes de chocar, una brújula giratoria, una barra que ve el piso y los interruptores que dicen "ya llegué".

Una cámara que mintió sobre su resolución

El 21 de octubre, en tu Jetson, pasó algo que le pasa a casi todos los que programan cámaras por primera vez. Abriste la cámara con `cv2.VideoCapture(0)` y OpenCV (la librería de visión por computadora de Python) te entregó un cuadro de **1536 × 2304** píxeles. Pero tu cámara USB **no tiene** ese modo; sus modos reales son 640 × 480, 1920 × 1080, etc. ¿Qué estaba pasando? Sin que se lo pidieras, OpenCV abrió la cámara por **GStreamer** (un sistema de tuberías de video de Linux) y este inventó un cuadro con un formato raro. Al forzar el backend **V4L2** (el controlador estándar de cámaras de Linux), llegó el cuadro correcto de **480 × 640**.

Esto está **VERIFICADO** por ti en la bitácora y fue la causa de que, en la primera prueba del backend en el Jetson, el dashboard transmitiera a 2304 × 1536 (¡el cuádruple de píxeles que lo necesario!) y se viera lento. La solución, ya en el repositorio desde el commit `c9f24cb`: variables de entorno opcionales `PERCEPTION_CAMERA_BACKEND`, `_FOURCC`, `_WIDTH` y `_HEIGHT` (y las equivalentes `LINE_VISION_CAMERA_*` de la celda) para forzar V4L2, MJPG y 640 × 480. Moraleja de cualquiera que trabaje con cámaras: **nunca asumas qué te entrega el sistema; pregunta y mide.**

La cámara del robot, en una frase y con sus datos

La **cámara** del diseño es una **Logitech 1080p** USB que "*ya la tiene Mario*", montada **arriba, al frente**, mirando hacia la banda. Su trabajo, según la hoja: **leer el QR de cada fixture** (el código viene en el travesaño trasero) y **los ArUco de las estaciones**, y aportar a YOLO (capítulo 14). En tu Jetson, esta cámara aparece así (**VERIFICADO**, salida reportada):

Dato	Valor
Nombre	"1080P Pro Stream" (puerto `usb-3610000.usb-2.3`)
Dispositivos	`/dev/video0` (captura) y `/dev/video1` (solo metadatos)
Identificador USB	`046d:0894` (en `lsusb` sale como Logitech CrystalCam; es la misma cámara)
Formatos	YUYV y MJPG; MJPG de 160 × 90 hasta 1920 × 1080 a 30 FPS

Comandos para repetir tú esa revisión, **en el Jetson, por SSH** (`[Jetson · SSH]`):

```
v4l2-ctl --list-devices
v4l2-ctl -d /dev/video0 --list-formats-ext
```

El primero lista las cámaras; el segundo, todos los modos que ofrece cada una.

FPS, exposición y el misterio de los cuadros que se caen

Un **FPS** (*frames per second*, cuadros por segundo) mide cuántas imágenes entrega la cámara cada segundo. Tu agente de percepción en el Jetson midió **29.0 FPS de cámara y 29.1 de entrega**, a 640 × 480, con una inferencia de YOLO de **68.0 ms** de promedio (**MEDIDO**, reportado por ti el 23 de octubre).

Pero antes, en una prueba con **poca luz** y la cámara sin acomodar, la captura bajó a **10 FPS** con Python y a **15.3 FPS** con `v4l2-ctl`. La razón es un clásico: cuando hay poca luz, la cámara **alarga el tiempo de exposición** (la cámara "mira" más tiempo cada cuadro para juntar más luz) y por eso entrega menos cuadros por segundo. La opción `exposure_dynamic_framerate` permite que la cámara baje los FPS automáticamente si hace falta luz. Al apagarla **solo como prueba**, el promedio subió pero no se estabilizó: no es una medición representativa. **No se fija nada de exposición desde el código** y la prueba buena con luz real y la cámara en su posición final queda **PENDIENTE (P-24)**.

Mensaje práctico: **la luz es un componente del sistema**, como la batería. Un robot que "anda perfecto en la mesa" y "se ve ciego" en el pasillo oscuro tiene un problema de iluminación, no de software.

ArUco y códigos: la señal que el robot ya sabe leer

Un **ArUco** es un cuadrado blanco y negro con un patrón único, como un código QR más simple, diseñado para que una cámara calcule **dónde está y hacia dónde mira**. Aether lo usa para **verificar su posición** al llegar a la banda y a la estación de descarga (el robot navega por **waypoints**, puntos de ruta, y el ArUco corrige el error acumulado). Los **códigos de barras/QR** se leen con la librería `pyzbar` y sirven para identificar cada fixture. El tamaño impreso de los marcadores y la distancia a la que deben leerse **no están en ningún documento**: **PENDIENTE (P-12)**.

Dato de tu propio proyecto para el capítulo de visión: el ArUco **puede dar un identificador incorrecto pero válido con reflejos de luz**, y el reporte lo documenta como hallazgo real (capítulo 14).

El bus I²C: seis dispositivos, dos cables

Varios sensores del diseño comparten un **bus I²C** (*Inter-Integrated Circuit*, "I cuadrada C"): **dos cables** (**SDA** para datos y **SCL** para el reloj) por donde el ESP32 habla con muchos dispositivos, cada uno con una **dirección** única. Es como un grupo de WhatsApp donde cada persona tiene su nombre. En tu diseño, los pines son el **GPIO 21 (SDA)** y el **GPIO 22 (SCL)** del ESP32, y las direcciones son:

Dispositivo	Dirección I²C	Qué hace
PCF8574 (expansor de pines)	`0x20`	Da 4 salidas extra para el truco de los VL53L0X
BNO055 (IMU)	`0x28`	Mide el giro del robot
VL53L0X frontal	`0x30`	Distancia al frente
VL53L0X lateral izquierdo	`0x31`	Distancia a la izquierda
VL53L0X lateral derecho	`0x32`	Distancia a la derecha
VL53L0X de la pila	`0x33`	¿Hay un fixture arriba de la torre?

La hoja pide una comprobación con un **escáner I²C** (un programita que pregunta a todas las direcciones quién contesta): "*El escáner I²C lo encuentra en 0x20*", etc. Si el escáner no ve un dispositivo, el problema es de cableado, no de programa.

VL53L0X: medir distancias con un rayo de luz

El **VL53L0X** es un sensor de **tiempo de vuelo** (*Time-of-Flight*, ToF): manda un pulso de luz infrarroja, espera a que rebote en un objeto y, con el **tiempo** que tardó, calcula la distancia (la luz viaja a 300 000 km/s: se necesita un reloj muy rápido). El diseño pone **cuatro**:

- **Tres de obstáculos** a unos **15 cm del piso**: uno al frente y uno a cada lado. Si ven algo a **menos de 40 cm**, **frenan el carrito sin esperar al Jetson**. Es una capa de seguridad **reactiva**: decide el ESP32 al instante, sin mandar nada por USB.
- **Uno arriba de la torre**, mirando hacia abajo, que **confirma que hay un fixture** en la pila (presencia, con ±25 mm).

Hay un problema típico de ingeniería, y el diseño lo resuelve con una idea elegante: **los cuatro sensores vienen con la misma dirección de fábrica**, así que en el mismo bus chocarían. La solución: cada sensor tiene un pin **XSHUT** (apagado). Al arrancar, el ESP32 los apaga a todos mediante el **PCF8574** (P0 a P3), los **prende uno por uno** y a cada uno le cambia la dirección a `0x30`, `0x31`, `0x32` y `0x33`. Es el equivalente de que cada persona entre al grupo de WhatsApp por separado y se ponga su nombre.

La hoja de armado pide verificar "*Mide bien a 10, 30 y 60 cm*" con una regla. Una advertencia práctica: **los cables de señal pegados a los de potencia hacen que los VL53L0X y el RS485 fallen "al azar"**. Alcance máximo y precisión del sensor son dato del fabricante (**conocimiento general**: del orden de un par de metros); no están medidos en tu proyecto.

BNO055: una brújula de giro (sin brújula)

El **BNO055** es una **IMU** (*Inertial Measurement Unit*, unidad de medición inercial): un chip con **acelerómetro** (mide aceleraciones), **giroscopio** (mide cuánto gira) y, en su modo completo, **magnetómetro** (mide el campo magnético, como una brújula). En el diseño sirve para **corregir la odometría** de las ruedas: las ruedas pueden patinar y creerse que giraron 90° cuando giraron 85°; el giroscopio no se deja engañar por eso.

Detalle de oficio: la hoja pide el **modo IMU, sin magnetómetro**, "*porque los motores y la batería lo confunden*": los imanes y las corrientes grandes alteran el campo magnético alrededor del robot. Comprobación del diseño: "*al girar el carrito 90° marca ~90°*". Y una nota de la hoja sobre la compra: el módulo que compraron trae sus 8 pines en una sola fila, distinto del modelo de Adafruit dibujado: **lean la serigrafía del módulo real** antes de soldar.

QTR-8A: una barra que ve el piso (de respaldo)

La **QTR-8A** es una barra con **8 sensores infrarrojos** que miran el piso y devuelven un valor **analógico** (un voltaje entre 0 y 3.3 V) según qué tan oscura sea la superficie: cinta negra sobre piso claro, por ejemplo. Aquí va **debajo del frente, a 3–5 mm del piso**, alimentada a **3.3 V (nunca 5 V)** y usa solo **5 de los 8** (las del centro: salidas 3 a 7, GPIO 32 a 36). Su papel es de **respaldo**: "*seguir una línea en el piso si la navegación por waypoints falla*". Es decir, un plan B.

Finales de carrera: "ya llegué" con un clic

Los **tres finales de carrera** son interruptores mecánicos que el carro pisa al llegar al límite. Dan el **cero** (*home*): el programa mueve el motor hasta que el interruptor se activa y a partir de ahí **cuenta pasos**. Van conectados como **contacto normalmente cerrado (NC)** con **pull-up** interno (una resistencia que "tira" el pin hacia 3.3 V): "*un cable roto se lee como 'presionado' y el motor se detiene*". Es la misma idea de **falla segura** del capítulo 4.

Los infrarrojos de la celda: tres hilos y mucho cuidado

En la celda hay **dos infrarrojos industriales** (IR1 en la estación de visión e IR2 después del pistón), pero de ellos habla el capítulo 7. Aquí solo conviene dejar la idea central del capítulo 2: **PNP o NPN**, y **24 V, no 5 V**.

Lo que no existe en este diseño (y vale decirlo)

- **Lidar:** la guía vieja contemplaba un lidar 2D en USB ("*cuando se compre*", y reconocía que "*no existe todavía el programa de ruta*"). **La hoja de armado rev. 3.8 no lo incluye.** Hoy la prevención de choques es con los tres VL53L0X y la navegación es por waypoints con ArUco. **NO VERIFICADO**: nadie confirmó si se compra un lidar: **PENDIENTE (P-11)**.
- **Micrófono y bocina del robot:** el micrófono de Lumina es el de la laptop (capítulo 15). Si el Jetson tendrá micrófono propio **no se sabe**: **PENDIENTE (P-22)**.
- **Sensores de la batería:** solo el divisor de voltaje del capítulo 4; **no hay medición de corriente**.

Calibración: el mundo físico no es ideal

Todos los sensores de este capítulo tienen un **error propio**: el VL53L0X lee unos milímetros de más o de menos según el color del objeto; el giroscopio "se desvía" lentamente con los minutos; la cámara cambia con la luz; las ruedas patinan. Por eso casi cada comprobación de la hoja termina en una medición con regla o con multímetro: "*Mide bien a 10, 30 y 60 cm*", "*marca ~90°*", "*5.0–5.1 V sin carga*". **Un buen programa deja perillas para ajustar** (offsets, ganancias, umbrales) en vez de asumir que el sensor es perfecto.

Qué hace la industria

Los robots móviles comerciales combinan **capas**: una **capa reactiva rápida** (sensores de distancia y bumpers que frenan al instante), una **capa de navegación** (mapas, **lidar**, odometría y cámara) y una **capa de seguridad certificada**. Tu diseño sigue esa lógica a escala de laboratorio: VL53L0X (reactiva), cámara + ArUco + odometría (navegación) y el paro físico (seguridad). Lo que cambia es que los productos usan **lidar de seguridad certificado**, con zonas de frenado configurables.

Para ponerte a prueba

1. ¿Por qué una cámara entrega menos FPS con poca luz, y qué cambia si pones una lámpara?
2. ¿Qué es una dirección I^2C y por qué los cuatro VL53L0X necesitan el truco del XSHUT?
3. ¿Qué ventaja tiene que el ESP32 frene el robot al ver algo a menos de 40 cm, en lugar de que lo decida el Jetson?
4. ¿Por qué se usa el BNO055 en modo IMU (sin magnetómetro) en un robot con motores y batería?
5. ¿Qué pasa con el valor de un final de carrera normalmente cerrado si se rompe su cable?
6. ¿Qué dato falta hoy para que el ArUco sirva de verdad: tamaño impreso, distancia o iluminación?

Capítulo 7: La celda de manufactura

Banda, fixtures, cámara, pistón y un controlador LOGO: qué hace cada cosa, qué dice el código (que es quien manda) y qué dicen los documentos viejos.

Tres versiones de la misma celda

Abre la carpeta `docs/hardware/` y lee la hoja de conexión de la celda. Te cuenta una historia: la Raspberry Pi 4 lee un **PLC** (*Programmable Logic Controller*, controlador lógico programable, el cerebro industrial de tableros) por **Modbus TCP**, los infrarrojos van al PLC, y "*no hay ningún hilo entre el PLC y los GPIO de la Pi*".

Ahora abre `backend/cell_line.py`, el código que de verdad corre. Su primera hoja dice otra cosa: la Pi enciende **dos relevadores** con sus propios pines (**GPIO 8** para el pistón y **GPIO 9** hacia la entrada **I4 de un LOGO**, un mini-PLC), y el LOGO es quien mueve la banda.

Y hay una tercera versión: el borrador de la ADR 0011 dice que la Pi "*no controla el pistón*". En la auditoría del 22 de octubre (`AUDITORIA_FASE_A_B.md`, hallazgo A6) quedó anotado que son **cuatro versiones** (código, hoja de conexión, ADR y lo que se dijo en una conversación) y que "*solo tu equipo puede decir cuál es la real*". Tú me diste una regla para este libro: **ante una contradicción entre documento y código, gana el código**. Así que este capítulo cuenta la celda **como la describe** `cell_line.py`, y deja anotado en `NOTAS_CORRECCIONES.md` lo que los documentos dicen distinto. Lo que está realmente cableado en la mesa **no lo ha confirmado nadie del equipo**: **PENDIENTE (P-13)**, con Uriel (comunicación de la Pi con el PLC y la visión), David (electrónica) y Pedro (pistón).

Quién lleva qué en la celda

Según me dijiste, las áreas del equipo se reparten así. Donde pido detalle físico lo dejo **PENDIENTE de confirmar con la persona**, sin inventar:

Persona	Área	Qué falta confirmar con esa persona
Mario	Todo el software del robot y la visión	—
Isaac	Diseño mecánico, CAD e impresión 3D (la banda y los fixtures)	Dimensiones oficiales y material de los fixtures (P-23)
Pedro	El pistón de scrap	Tipo de pistón, electroválvula, tiempos reales (P-15)

Persona	Área	Qué falta confirmar con esa persona
Uriel	Comunicación de la Raspberry con el PLC y la visión	Cableado real: GPIO + LOGO o PLC por Modbus (P-13)
David	Ensamble mecánico y electrónica	Relevadores, fuente, paro de emergencia (P-17, P-18)
Emmanuel	HMI/GUI aparte	Alcance de la pantalla de la celda (P-19)

La cadena física, de la banda al scrap

La idea, que viene de la ADR 0010: una **banda transportadora** lleva piezas, cada una sobre un **fixture** (la base que la sostiene en posición). Frente a la **cámara** la pieza se detiene un instante; la **visión** decide si es **PASS** (pieza real registrada) o **FAIL** (no lo es); si es FAIL, un **pistón** la empuja fuera de la banda (*scrap*, desecho); si es PASS, la banda sigue y la recoge el robot.

Lo que dice el código sobre quién hace qué:

- **La Raspberry Pi 4** corre `celda_agent.py`: abre la cámara, corre el modelo de visión, decide y **enciende los dos relevadores** (pistón y banda).
- **El LOGO** (mini-PLC) mueve la banda. `cell_line.py` lo dice así: "*El LOGO, no este archivo, ignora el infrarrojo 5 s al ver I4*". O sea: el LOGO tiene un infrarrojo conectado a sí mismo, y cuando la Pi manda el pulso a su entrada **I4**, el LOGO avanza la banda e **ignora el sensor durante 5 s** para no volver a detectar la misma pieza. (La marca y modelo exactos del LOGO no están escritos: **PENDIENTE (P-14)**.)
- **El pistón** es un cilindro neumático (mueve un émbolo con aire comprimido) accionado por una **electroválvula** que abre el aire cuando su **relevador** recibe el pulso de la Pi.

Qué decide la visión: un PASS que no es lo que parece

Antes de seguir, una honestidad que el propio código declara. En `backend/dataset_store.py`, la función `clasificar_pieza` dice en su descripción: "*PASS significa que el nombre está en clases, no que la pieza esté libre de defectos.*" Es decir:

- El modelo reconoce **tres piezas registradas**: `computadora1`, `computadora2` y `computadora3` (**VERIFICADO**, archivo `classes.txt`). Son las **ECU** (computadoras de motor) que viajan en los fixtures.
- **PASS** = el modelo vio una de esas tres con **confianza ≥ 0.50** y sin ambigüedad (la segunda candidata queda al menos **0.10** por debajo).

- **FAIL** = el modelo vio algo, pero **no es una pieza registrada** (o dudó entre dos).
- **SIN_PIEZA** = el modelo no vio nada: no hace nada.

No hay inspección de defectos todavía. Una ECU rayada o con un conector roto pasaría como PASS. El criterio real de calidad es una decisión pendiente de diseño: **PENDIENTE (P-26)**. Esto importa para el jurado: lo honesto es decir "*la celda distingue piezas registradas de objetos que no lo son*", no "*la celda inspecciona calidad*".

Cifras del modelo (**MEDIDO**, `parts_report.json`, entrenamiento del 15 de octubre de 2026): **mAP50 = 0.995**. El **mAP50** es una nota de qué tan bien el modelo localiza y acierta objetos (1.0 es perfecto). Con un dataset tan chico (el repositorio tiene 19 fotos en la bandeja de entrada), esa nota es **optimista**: el modelo no ha visto la variedad de ángulos, luces y fondos del mundo real. Tu bitácora midió además que el modelo exportado a **ONNX** juzgó **igual que el original en 19 de 19 fotos** (IoU medio 0.994), pero que **no se probó con objetos que no sean pieza**, así que el FAIL real **no está medido** con este modelo.

Qué pasa con una pieza, segundo a segundo

Los números salen de las variables de entorno **por defecto** de `cell_line.py` (**VERIFICADO** en código; **NO MEDIDO** en una Pi real):

1. **La visión juzga cada 0.5 s** (`LINE_ANALISIS_CADA_S`) un solo cuadro con el modelo ONNX.
2. **Se exigen 5 juicios iguales seguidos** (`LINE_ESTABLES`) antes de actuar. Así un parpadeo de luz no dispara el pistón. Tarda unos 2.5 s más lo que dure cada análisis. La documentación avisa: "*No bajar de 3 sin probar: un FAIL falso dispara el pistón.*"
3. **Una pieza dispara una sola vez**: tras actuar, el sistema se "desarma" hasta que ve **SIN_PIEZA**; así la misma pieza no se juzga dos veces.
4. **Si es FAIL:** espera **3 s** (`LINE_PISTON_ESPERA_S`: lo que tarda la pieza en llegar frente al pistón), **enciende el pistón 1 s** (`LINE_PISTON_ON_S`), lo apaga y espera **2 s** a que **regrese** (`LINE_PISTON_REGRESO_S`).
5. **Después, pulso de banda de 0.5 s** (`LINE_BANDA_ON_S`) por el GPIO 9: avanza la banda. En **PASS** este pulso sale **al terminar el análisis**, sin esperar al pistón.
6. Se anota el evento: `scrap_expulsado` (FAIL con todo en orden) o `pieza_aceptada` (PASS con todo en orden).

Un FAIL completo suma, con los valores por defecto: **≈ 2.5 s de estabilización + 3 s + 1 s + 2 s + 0.5 s ≈ 9 s** por pieza (estimación a partir de las constantes; **NO MEDIDA**). Todos esos tiempos se calibran **en la banda real** (**PENDIENTE (P-15, P-16)**); los números del código son puntos de partida, no la verdad.

Los estados de una pieza, en los datos

Cada pieza deja un registro con tres campos: `resultado` (**PASS** o **FAIL**), `accion` y la hora. Las acciones posibles que escribe el código:

Acción	Cuándo ocurre
`pieza_aceptada`	PASS y el pulso de banda salió
`scrap_expulsado`	FAIL, el pistón disparó, regresó, y la banda avanzó
`fallo_actuacion`	FAIL pero algo falló al actuar (por ejemplo, un pulso no pudo enviarse)
`pulso_no_enviado`	PASS pero el pulso de banda no pudo enviarse

Dos filosofías que enseña esta tabla: **no se finge** (si no hay forma de mover el relevador, el registro lo dice) y **un FAIL nunca se reporta como aceptado** (si la visión dijo FAIL y el pistón no actuó, es `fallo_actuacion`).

La regla de seguridad que protege contra errores: sin `gpiod`, nada se mueve

`cell_line.py` solo mueve un pin si puede importar la librería `gpiod` (la que habla con los pines del chip en Linux moderno). Si no la encuentra, `poner()` devuelve `False` sin tocar nada y el código **no finge** que algo se movió. **Hoy esa librería no está instalada** ni en el Jetson (**VERIFICADO**: `python -c "import gpiod"` falla) ni se ha probado en la Pi. Es una protección involuntaria buena: mientras no se instale, ningún botón de la pantalla puede mover un pin.

La bitácora de la sesión 5 lo verificó leyendo el código y reproduciéndolo con las clases reales en la laptop: **ARRANQUE y PARO solo cambian el interruptor interno** (`running`), PARO además **aborta el ciclo en curso y apaga las salidas**, y el único camino a un pin es `celda.ver(juicio)` disparado por **cinco juicios de cámara**, nunca por un botón. La regla del proyecto, escrita en `docs/guia_arranque.md`: "*No instalar `python3-libgpiod` hasta confirmar el cableado.*" Antes de instalarlo (capítulo 8) hay que medir con LED y multímetro.

Una trampa de nombres que ya te confundió: en el JSON de estado, `sources.plc` **no significa que haya un PLC**: es `salidas.abiertas`, es decir, "¿se abrió el chip GPIO?". Sin `gpiod`, siempre es `false`, y por eso aparece la alarma **"Enlace PLC perdido"** en la pantalla (la bitácora la marcó como *esperada: no hay PLC*). Es un nombre heredado de la versión PLC del diseño.

Estados, alarmas y sesión: la pantalla de la celda

La página `celda.html` (que sirve **la propia Pi**, no la laptop) muestra el estado de la celda. El **estado** (`state`) puede ser:

- **OPERANDO**: la celda está activa, la cámara viva y hay un ciclo permitido.
- **DETENIDA**: o el operador pulsó **Paro** ("Paro solicitado por el operador") o la cámara no ha juzgado ningún cuadro en los últimos **2 s** ("Cámara apagada").

(Una tercera, `SIN_CONEXION`, existe en la hoja de conexión del PLC, pero el código de `cell_line.py` no la genera.)

`cell_hmi.py` añade **alarmas** al estilo de la norma **ISA-18.2** (la que ordena cómo se muestran y reconocen las alarmas en planta): cada una con **prioridad, causa y acción**, y **viven en el servidor** (sobreviven si recargas la página). Hay tres: "controlador no responde", "visión en FAIL y el pistón no disparó" y "detenida". La **sesión** usa una cuenta única de laboratorio (norma **IEC 62443** como referencia): quien manda el ciclo debe haber iniciado sesión. **La contraseña ya no está en el código** (antes era `admin` / `admin`); se toma de la variable de entorno `CELDA_HMI_CLAVE`. Sin esa variable *nadie* puede iniciar sesión y Arranque/Paro no responden. Hallazgo real de `iniciar_aether.ps1 -SoloComprobar`: tu `backend\.env` **todavía no la tenía definida**. El módulo mismo avisa de su alcance: "*No es una certificación del sistema. El paro de emergencia no pasa por aquí.*"

Quién diseña la pantalla y su interfaz final es **Emmanuel**: lo que existe hoy es la que escribió el software; el alcance de la HMI "aparte" es **PENDIENTE (P-19)**.

"La celda se opera sola"

Me pediste que se entendiera esto: **la laptop es el servidor central, pero la celda tiene su propio arranque y paro y debe poder operarse sola.** Eso está respaldado por el diseño del código:

- `celda_agent.py` arranca **solo la celda** (cámara, visión, pulsos, alarmas y `celda.html`); **no importa** Lumina, la base de datos, `torch` ni `ultralytics`. Hay una prueba que lo comprueba.
- Sirve **su propio video** (`/line/vision/stream`) y su **propia página**; el video **no pasa por la laptop**.
- **Decide y actúa aunque la laptop o la red se caigan.** A la laptop solo le manda **eventos** (`POST /line/cell/events`) con un identificador único por pieza, en una **cola con reintento**; allá solo se **anotan**, no accionan nada.
- Los botones **Arranque y Paro** de la pantalla viven en la Pi (cuando hay sesión).

Lo que **no existe** en los documentos: un **arranque o paro físico** (botón de tablero) independiente de la pantalla. Si la celda debe poder operarse sola *sin pantalla*, hace falta un botón cableado y, sobre todo, el **paro de emergencia físico**: **PENDIENTE (P-17)**.

La Raspberry Pi 4 corre la visión con un truco para no ahogarse

Una Pi 4 es mucho más modesta que una laptop. Para no saturarla, `celda_agent.py` hace tres cosas, medidas en la **laptop** (no en la Pi):

- Un **solo modelo** en lugar de dos (el modelo de entorno `yolov8n` se quita).
- **Un análisis cada 0.5 s** en vez de cada cuadro.
- El modelo corre en **ONNX** con `cv2.dnn` de OpenCV, sin `torch` (que pesa mucho).

Resultado (**MEDIDO en la laptop**, 8 s con cámara, sin nadie viendo el video): el pipeline anterior de `main.py` gastaba **381 % de un núcleo** con unos 6 análisis/s; el agente nuevo, **18 %** con 2 análisis/s. **Falta medirlo en la Pi real**: **PENDIENTE (P-20)**. Un hallazgo curioso de la medición: con DirectShow (Windows) `cap.grab()` no espera al siguiente cuadro y daba 1.5 millones de vueltas por segundo, ocupando un núcleo entero; con `read()` sí espera.

El PLC y Modbus: lo que quedó en papel

La hoja de conexión del 5 de octubre y la guía describen un mapa de **señales Modbus** (`rechazar`, `banda_marcha`, `rpi_vivo`, `plc_vivo`, `accion`, `pieza_id`) con direcciones de ejemplo y una columna "Real" **vacía**. **Modbus TCP** es un protocolo industrial para leer y escribir registros de un PLC por Ethernet (puerto 502). La hoja misma dice que la carpeta `edge/manipulation/line_actuation/plc_backend/` está **vacía a propósito** hasta llenar esa columna, y que **el código no consume** `CELL_AGENT_URL`. Es decir, el mapa de Modbus

es un **plan de comunicación para la versión con PLC**, no lo que corre. Si en el taller se termina cableando un PLC en lugar de GPIO + LOGO, ese mapa vuelve a ser el punto de partida. Decisión de Uriel y David: **PENDIENTE (P-13)**.

Qué dicen los documentos viejos y el código

Tema	Documento viejo	Código real (gana)
Quién enciende el pistón	PLC por Modbus; la Pi no lo energiza	La Pi, por GPIO 8 y un relevador
Quién mueve la banda	PLC	LOGO, avisado por GPIO 9 → entrada I4
Criterio PASS/FAIL	"Hoy solo detecta presencia" (ADR 0010, hoja de conexión)	Ya distingue piezas registradas de otros objetos (`clasificar_pieza`); defecto: no
`tipo_objeto` (1/2/3)	"No existe" (ADR 0010)	Existe: es la posición de la clase en `classes.txt` + 1
Dónde se ve `celda.html`	La laptop	La propia Pi
Modelo del pistón	"PENDIENTE"	Sigue pendiente de datos

Cómo probar la celda sin riesgo

1. En la laptop, **sin `gpiod`**: la celda corre completa, la pieza se juzga y se anota como `pulso_no_enviado` o `fallo_actuacion`. **Nada se mueve.**
2. Corre las pruebas: `[Laptop · PowerShell]`, desde la raíz del proyecto: `backend\.venv\Scripts\python.exe -m pytest tests\backend\test_cell_line.py tests\backend\test_celda_agent.py -q`. (No ejecuté este comando al escribir el libro; las pruebas de `cell_line` usan una espera falsa y relevadores falsos, según su código.)
3. Para la Pi, **primero** LED y multímetro (capítulo 8), **después** instalar `python3-libgpiod`.

Qué hace la industria

Las celdas industriales de inspección usan **PLC certificados** para la lógica de seguridad y movimiento, y una **PC o cámara inteligente** para la visión; se comunican por **Modbus, PROFINET o EtherNet/IP**, y el **paro de emergencia es un relevador de seguridad** cableado, nunca una pantalla. Tu celda sigue ese principio: la lógica de visión corre en la Pi y la potencia se maneja por relevadores y un LOGO.

Para ponerte a prueba

1. ¿Por qué `PASS` en tu celda no significa "sin defectos"? ¿Qué diría el jurado si lo afirmaras?
2. ¿Qué evita que un parpadeo de luz dispare el pistón?
3. Con los valores por defecto, ¿cuánto dura un FAIL completo y qué parte se afina con la banda real?
4. ¿Por qué `sources.plc` es `false` aunque no haya ningún PLC, y qué alarma provoca?
5. ¿Qué diferencia hay entre el botón "Paro" de la pantalla y un paro de emergencia?
6. ¿Qué cuatro versiones del cableado existen hoy y quién del equipo puede decir cuál es la real?

Capítulo 8: La Raspberry Pi 4 y la seguridad eléctrica del GPIO

Para qué sirve la Pi de la celda, cómo se instala sin torch, por qué un pin puede disparar el pistón al encender y qué se mide antes de dejar que el código toque algo real.

Una computadora del tamaño de una tarjeta, con pines que tocan el mundo

La **Raspberry Pi** es una computadora completa en una placa del tamaño de una tarjeta: procesador tipo móvil (ARM), memoria, puertos USB y de red, ranura para tarjeta **microSD** (donde vive el sistema) y, en un costado, **40 pines** llamados **GPIO** (*General Purpose Input/Output*, entrada y salida de propósito general) con los que puede leer un botón o encender un relevador. Corre **Linux** (el sistema operativo oficial se llama *Raspberry Pi OS*; la versión actual se basa en **Debian 12 "Bookworm"**).

En Aether hay **dos modelos mencionados** y conviene separarlos desde ya:

Raspberry	Papel	Estado
Raspberry Pi 4	Computadora de la **celda de manufactura**, junto a la banda: cámara de línea, visión, relevadores, `celda.html`	**Vigente**, pero **sin probar** en el hardware real (**PENDIENTE (P-20)**)
Raspberry Pi 5 con AI HAT+	Se pensó para el robot (plan del 30 de septiembre)	**Descartada** (me lo confirmaste); la guía del 9 de octubre que la describe quedó vieja

Tampoco la usas como cerebro del robot: ese es el **Jetson** (capítulo 9). El libro piloto ya explicó qué es el GPIO y por qué un motor no se conecta directo a un pin; aquí **no lo repito**: ve al capítulo 2 (voltaje, corriente, niveles lógicos) y amplío con lo que el código y la hoja de conexión nos dan.

Instalar la celda en la Pi: sin torch, sin ultralytics

El agente de la celda (`backend/celda_agent.py`) está pensado para que la Pi **no** cargue Lumina, ni la base de datos, ni `torch` ni `ultralytics`: son pesadísimos. El modelo de visión corre en **ONNX** (un formato neutral de modelos) con `cv2.dnn`, un módulo de OpenCV. El archivo `backend/requirements-pi.txt` lo dice con todas sus letras:

> Se instala en un venv con `--system-site-packages`: OpenCV y NumPy salen de `apt` (`python3-opencv`, `python3-numpy`), **no de pip**. Prohibido en la Pi: `opencv-python`, `opencv-contrib-python`, `torch` y `ultralytics`. Requiere Python ≥ 3.10 (Raspberry Pi OS Bookworm trae 3.11).

¿Por qué `--system-site-packages`? Un **venv** (entorno virtual de Python) normalmente es una caja aislada; esta opción le permite **ver** los paquetes instalados por `apt`. Así la Pi usa el OpenCV compilado para ella (rápido), y pip instala solo `fastapi` y `uvicorn`.

Pasos (copiados de `COMANDOS_LINUX.md`, sección 7; ninguno corrido todavía en la Pi)

Una vez, en la laptop (el modelo se exporta aquí porque la Pi no tiene torch) — `[Laptop · PowerShell]`, desde la raíz del proyecto:

```
edge\.venv\Scripts\python.exe -m pip install onnx
$env:YOLO_AUTOINSTALL = "False"
edge\.venv\Scripts\python.exe -c "from ultralytics import YOLO;
YOLO('edge/perception/weights/parts.pt').export(format='onnx', imgsz=320,
opset=12)"
```

Luego copia el archivo a la Pi (`<usuario>` y `<ip-pi>` son los de tu Pi, que **no están en el repositorio**) — `[Laptop · PowerShell]`:

```
ssh <usuario>@<ip-pi> "mkdir -p ~/aether/edge/perception/weights"
scp edge\perception\weights\parts.onnx <usuario>@<ip-
pi>:~/aether/edge/perception/weights/
```

Una vez, en la Pi — `[Pi 4 · SSH]`:

```
sudo apt install -y git python3-venv python3-opencv python3-numpy
git clone --depth 1 --filter=blob:none --sparse <url-del-repo> ~/aether && cd
~/aether
git sparse-checkout set backend dashboard edge/perception/parts
python3 -m venv --system-site-packages ~/celda-venv
~/celda-venv/bin/pip install -r backend/requirements-pi.txt
```

El `git sparse-checkout` baja **solo las carpetas necesarias** (`backend`, `dashboard` y las clases del modelo), no todo el repo.

Arrancar (dentro de `tmux`, para que cerrar la ventana de SSH no mate el programa) — `[Pi 4 · SSH]`:

```
cd ~/aether/backend && ~/celda-venv/bin/python -m uvicorn celda_agent:app --host
0.0.0.0 --port 8000
curl -s localhost:8000/health   # esperado: {"status":"ok","error":null}
```

Después, desde la laptop: `http://<ip-pi>:8000/celda.html` en el navegador. Si `health` falla, el campo `"error"` dice por qué.

Una incógnita técnica real

Tu modelo ONNX se probó con **OpenCV 5.0** en la laptop. **Bookworm trae OpenCV 4.6.** Si `cv2.dnn` de esa versión no sabe leer el ONNX (por un operador que cambió entre versiones), la celda no arranca. La bitácora ya lo marcó: **NO VERIFICADO**, y la primera prueba en la Pi debe incluir `top` (para ver CPU) y `curl /line/cell/status` con una pieza delante. Qué sistema, qué RAM y qué cámara tiene exactamente tu Pi 4: **PENDIENTE (P-20)**.

Los 40 pines: dos formas de contarlos

Una confusión clásica del GPIO: **cada pin tiene dos números**.

- El **número físico**: la posición en el conector, del 1 al 40.
- El **número BCM** (por el chip de Broadcom): el nombre "lógico" del GPIO. Es el que usa el software.

`cell_line.py` usa números **BCM**: **GPIO 8** para el pistón y **GPIO 9** para la banda. En el conector físico, esos corresponden (**regla general, verifícalo en el diagrama oficial `pinout.xyz` antes de cablear**) a los pines **24 y 21**. Los nombres se pueden cambiar con `LINE_GPIO_PISTON` y `LINE_GPIO_BANDA`. La bitácora señala un riesgo derivado: esos números son **BCM de Raspberry Pi**; en el **Jetson** serían **otras líneas** del chip, así que **jamás** debe instalarse `gpiod` en el Jetson (hay una comprobación en los scripts de arranque que se niega a seguir si lo ve).

El estado del pin al encender: la trampa que no se ve en el código

Esta es la lección más valiosa del capítulo, y es **regla general** (no medida en tu Pi): al **encender** una Raspberry, **antes** de que tu programa corra, los pines no están "apagados": cada uno tiene una resistencia interna que lo **jala** hacia 3.3 V (*pull-up*) o hacia 0 V (*pull-*

down). Según la documentación del chip, los **GPIO 0 a 8 arrancan con pull-up** (nivel alto) y los GPIO 9 a 27 con pull-down (nivel bajo). Aplicado a tu celda:

- **GPIO 8 (pistón)** arrancaría en **alto**.
- **GPIO 9 (banda)** arrancaría en **bajo**.

¿Y qué hace una placa de relevadores con un pin en alto? Depende de **cómo esté hecha**: muchas placas baratas son **"activas en bajo"** (el relevador se enciende cuando el pin está en **bajo**), otras **"activas en alto"**. Con una placa activa en alto, **el pistón se dispararía solo al encender la Pi**. Qué tipo de placa tiene el equipo y cómo está alimentada la bobina **no está escrito en ningún lado**: **PENDIENTE (P-18)**. Por eso el plan de seguridad (libro piloto, capítulo 10) exige medir **el estado de los pines al arrancar con un LED o un multímetro, sin carga conectada**, antes de conectar el pistón real.

Qué hay entre un pin y el pistón

Un pin de 3.3 V no puede mover una electroválvula de 24 V. La cadena segura es:

1. **Pin GPIO (3.3 V, pocos miliamperios)** →
2. **Módulo de relevadores** (entrada aislada con **optoacoplador**, idealmente, que separa eléctricamente el mundo de la Pi del mundo de 24 V) →
3. **Contacto del relevador** (aguanta los 24 V y la corriente) →
4. **Electroválvula del pistón**, con su **diodo de protección** en paralelo con la bobina si es de corriente continua. Ese diodo (*flyback*) absorbe el pico de voltaje que genera una bobina al apagarse; sin él, ese pico puede dañar el contacto del relevador.

La Pi se alimenta con **su propia fuente de 5 V** (la oficial es de 5 V y 3 A por USB-C), separada de los 24 V. Una Pi con poca energía muestra un aviso de **bajo voltaje** y puede comportarse raro: ya viste lo caros que son los errores "al azar".

Lista antes de energizar algo real

Basada en tu plan y en la regla del repositorio:

1. **Sin cargas.** Pi encendida, pines libres. Con multímetro, mide el voltaje de los GPIO 8 y 9 **al encender** y anota los valores.
2. **Un LED con resistencia** (capítulo 2) en lugar del relevador. Verifica que parpadea cuando el código lo pide, y que **no** parpadea al arrancar.
3. **Probar** `cell_line.py` con `gpiod` instalado pero **sin carga real**: ¿qué hace al detectar `FAIL`?

4. **Relevador sin pistón.** Escucha el clic y mide su contacto.
5. **Solo entonces** el pistón, con aire a presión baja y **sin pieza**.
6. **Paro de emergencia físico** cableado **antes** de cualquier actuador real (**PENDIENTE (P-17)**).
7. **Instalar** `python3-libgpiod` (`sudo apt install python3-libgpiod` y el usuario en el grupo `gpio`) **solo cuando el equipo haya confirmado el cableado** (**PENDIENTE (P-13)**).

Arrancar la celda sola, sin que nadie abra una ventana

Me dijiste que la celda "*debe poder operarse sola*". Eso significa que, al darle corriente a la Pi, el agente tendría que **arrancar solo**. Hoy se arranca a mano dentro de `tmux`. La decisión de arrancar con **systemd** (el sistema de Linux para servicios que arrancan solos) se descartó para el Jetson en la ADR 0012 *("otra pieza que mantener")*, y se anotó como posible mejora cuando el agente resulte estable. Para la celda, que se espera que funcione sin supervisión, vale la pena reconsiderarlo: **PENDIENTE (P-27)**. Mientras tanto, conviene saber que el agente reabre la cámara solo si se cae y no necesita que alguien abra el video.

La Raspberry Pi 5 de la guía vieja

Para que no te confunda cuando la veas en los documentos: la guía del 9 de octubre describe una **Pi 5** (8 GB) con fuente de 27 W, una pantalla HDMI, un ESP32 conectado por serial UART (activando `dtparam=uart0=on` en `/boot/firmware/config.txt`) y un navegador mostrando la cara de Lumina. Todo eso **corresponde al plan descartado**. No es necesario para el robot, que usa el Jetson y el ESP32 por USB. El **problema abierto del navegador del Jetson** (capítulo 9) es justamente la contraparte de esa decisión.

Qué hace la industria

En planta, un **Raspberry Pi** se usa para prototipos, visión y supervisión; los productos serios emplean **versiones industriales** (con caja, riel DIN y alimentación de 24 V, como las que venden fabricantes de computadoras de borde) y **nunca** como única barrera de seguridad: eso se deja a **relevadores de seguridad** y **PLC certificados**. Decirle esto al jurado, con estas palabras, demuestra criterio.

Para ponerte a prueba

1. ¿Por qué la Pi no lleva `torch` ni `ultralytics`, y qué usa en su lugar?
2. ¿Qué diferencia hay entre el número físico y el número BCM de un pin?
3. ¿Qué podría pasar al encender la Pi si la placa de relevadores es "activa en alto" y GPIO 8 arranca en alto?
4. ¿Para qué sirve el diodo en paralelo con la bobina de una electroválvula?
5. Escribe, en orden, los cuatro primeros pasos de la lista "antes de energizar algo real".
6. ¿Qué parte de `COMANDOS_LINUX.md` sección 7 ya se corrió en la Pi y cuál no?

Capítulo 9: El Jetson Orin Nano Super

El cerebro del robot, explicado desde cero: qué es, cómo se le instaló PyTorch con GPU, qué problemas reales dio el 21 y el 23 de octubre y cómo se diagnosticó cada uno.

Una computadora con una tarjeta gráfica pegada

El **Jetson Orin Nano Super** (de la empresa NVIDIA) es una computadora del tamaño de una cartera que lleva, en el mismo chip, un procesador y una **GPU** (*Graphics Processing Unit*, unidad de procesamiento gráfico). Una GPU es un chip con **miles de núcleos pequeños** que hacen la misma operación a la vez; sirve para pintar videojuegos y, por la misma razón, para multiplicar las enormes tablas de números que usan las redes neuronales. Es la pieza que permite que **YOLO** (el modelo que detecta objetos, capítulo 14) corra cuando el robot se mueve, sin depender de internet.

Los datos de **tu** Jetson (todos **reportados por ti** con salidas reales, `docs/jetson/SETUP_JETSON.md`):

Dato	Valor
Equipo	Jetson Orin Nano Super 8 GB, nombre de máquina `mario-desktop`, usuario `mario`
Sistema	Ubuntu 22.04, kernel `5.15.148-tegra`, arquitectura `aarch64` (ARM de 64 bits)
Jetson Linux	L4T R36 revisión 4.7 (es lo que el equipo llama "JetPack 6")
Python	3.10.12
CUDA / cuDNN / TensorRT	12.6 / 9.3 / 10.3
OpenCV del sistema	4.8.0 (con GStreamer 1.20.3)
Memoria	7.4 GiB de RAM, 3.7 GiB de swap, disco de 47 GB (≈ 23 GB libres)
Modo de energía	**25 W** (ID 1). Existen 7 W, 15 W, 25 W y `MAXN_SUPER` (el máximo, **no usado**)

Algunas siglas, para que no queden como magia:

- **CUDA** es el lenguaje y el conjunto de herramientas con que los programas "hablan" con la GPU de NVIDIA.
- **cuDNN** es una biblioteca de operaciones de redes neuronales optimizadas sobre CUDA.

- **TensorRT** es un "compilador" que reorganiza un modelo para correr más rápido en el Jetson. **No se usa todavía**: la decisión de adoptarlo se dejó para después de medir (ADR 0012).
- **JetPack / L4T** (*Linux for Tegra*): el paquete de sistema operativo y controladores de NVIDIA para Jetson.

Por qué se anota el modo de energía en toda medición: el mismo Jetson rinde distinto a 7 W que a 25 W. Una cifra de FPS sin su modo es una cifra que nadie puede comparar. Está escrito en la bitácora: "*toda medición de FPS debe anotar el modo de* `nvpmodel`".

Qué hace el Jetson en Aether (y qué no)

Según la ADR 0012 (**propuesta implementada en la rama `jetson-agente`, aún no aceptada por Mario**), el Jetson ejecuta un **agente**: un programa (`backend/aether_agent.py`, puerto 8001) que **abre la cámara del Jetson**, corre YOLO + lectura de códigos + ArUco, **sirve el video anotado** y manda a la laptop **solo observaciones en JSON**, cada una con un identificador único. El **video nunca viaja por WiFi**: lo que viaja son datos chicos. La **laptop** sigue siendo el servidor central (base de datos, Lumina). Esa filosofía —*mover datos chicos, no datos grandes*— es la del capítulo 1. El Jetson **no** es el que ejecuta la voz de Lumina (el micrófono es el de la laptop) ni mueve motores directamente: eso lo hace el **ESP32** (capítulo 5), al que le manda órdenes por **USB**.

Tabla de acontecimientos: la bitácora, como un cuento policíaco

Entre el 21 y el 23 de octubre el equipo preparó el Jetson y, en cada paso, apareció un problema real. Esta es la mejor escuela de diagnóstico que tiene tu proyecto.

Caso 1: el SSH que se cortaba solo (21 de octubre)

Síntoma: las sesiones de SSH (la conexión remota de terminal) se cortaban con `Connection reset` mientras el Jetson tenía **WiFi y cable Ethernet a la vez** en la misma red. **Hipótesis (sin comprobar):** con dos interfaces en la misma subred, el equipo tiene **dos IP** (`.18` y `.19`) y la ruta o el nombre `mario-desktop.local` pueden "saltar" de una a otra a mitad de sesión. **Qué se hizo:** se apagó el WiFi (`nmcli radio wifi off`) y se dejó solo el cable. Para sobrevivir a cortes de todos modos se usa `tmux` (un programa que mantiene tus ventanas de terminal vivas aunque se caiga la conexión). **Estado:** la causa **no está diagnosticada** y la estabilidad con solo el cable **no está verificada** en uso largo. **PENDIENTE (P-28).**

Caso 2: una librería que PyTorch pedía y NVIDIA no explicaba (21 de octubre)

Síntoma: al planear la instalación de PyTorch de NVIDIA, la documentación mencionaba `libcusparseLt.so.0` y un script que **no cubre CUDA 12.6**. **Qué se hizo:** en vez de adivinar, se buscó el archivo oficial para Jetson (`linux-aarch64`, versión **0.6.2.3**; la 0.7.0 exige CUDA 12.8). Se descargó, se **comprobó su huella** (SHA-256: igual a la calculada en la laptop), se verificó que era un binario ARM de 64 bits y se copió a `/usr/local/cuda/lib64`. Prueba: `ctypes.CDLL('libcusparseLt.so.0')` imprime "carga OK". **Límite honesto:** solo se probó que **se carga**, no que PyTorch funcione con ella (eso vino en el caso 4). **Nota de seguridad que se encontró:** `cp -a` conserva el dueño del archivo (`mario`), y una librería del sistema que un usuario común puede reescribir es un riesgo; se propuso `sudo chown root:root` (estado: **NO verificado** que se aplicara).

Caso 3: una dependencia que arrastraba el torch equivocado (21 de octubre)

Síntoma: al simular la instalación de `requirements-jetson.txt`, `pip` traía `torch` 2.14.1 **de PyPI**, que **no** tiene soporte de GPU de NVIDIA, a través de un paquete llamado `ultralytics-thop`. **Por qué importa:** habría **pisado** el torch bueno. **Corrección:** se sacó `ultralytics-thop` del archivo de requisitos y se instaló aparte con `pip install --no-deps` (sin dependencias). Resultado verificado con `pip --dry-run`: 43 paquetes, **ninguno** `torch`, `torchvision` ni `opencv`, y `numpy` 1.26.4 (menor que 2). **Regla que sale de ahí: cuando instales algo con `pip` sobre un entorno delicado, simula primero (`--dry-run`) y mira qué quiere cambiar.**

Caso 4: PyTorch con GPU funciona (21 de octubre)

Se instalaron las "ruedas" (*wheels*, paquetes precompilados) de NVIDIA: `torch 2.5.0a0+872d972e41.nv24.08` y `torchvision 0.20.0a0+afc54f7`. Resultados **reportados por ti**:

Prueba	Resultado
`torch.cuda.is_available()`	`True`
Dispositivo	Orin
`torch.version.cuda`	12.6
Multiplicación de matrices 512×512 en GPU	resultado finito (correcto)
¿`pip` cambió el torch por el de PyPI?	No: conserva el sufijo `nv`

Y una detección con YOLOv8n sobre una imagen de archivo (`device=0`): detectó `bus`, 4 `person` y `stop sign` en **46 ms** (modo 25 W). **Límite:** una sola medición con imagen estática; no son FPS y no sirve para los umbrales de aceptación.

Dos detalles de oficio que ahorran tardes:

- `YOLO_AUTOINSTALL=False` es obligatorio: sin él, `ultralytics` intenta instalar `opencv-python` al no ver los metadatos de pip del OpenCV del sistema, y rompe tu entorno.
- Si ves `Uninstalling torch` o `Collecting torch==…` mientras instalas, **cancela con `Ctrl+C`** antes de que diga `Successfully installed`.

Un dato que la bitácora anota con honestidad: describiste esa rueda como compilada para CUDA 12.4, pero el sistema dice 12.6; **no se verificó con qué CUDA se compiló** esa rueda. Y un hilo de foro de NVIDIA reporta, en L4T 36.4.7, el error "unable to allocate CUDA0 buffer" tras actualizar paquetes de Ubuntu; por eso **no se hizo `apt upgrade`** y no se probó con YOLO bajo carga larga: **NO VERIFICADO**.

Caso 5: una cámara con una resolución que no existe (21 de octubre)

Ya lo viste en el capítulo 6: OpenCV por GStreamer entregaba 1536 × 2304. Solución: forzar V4L2 con variables de entorno. Estado: **PENDIENTE** de pegar el log que confirma 640 × 480 (se reinició el backend con esas variables, pero no se pegó la línea que lo demuestra).

Caso 6: el backend corre en el Jetson… pero no debía (21 de octubre)

Como **prueba**, se corrió el backend completo en el puerto 8001. Funcionó: `/health`, `/line/cell/status` y `/inventory` respondieron. Pero no era la arquitectura prevista (el backend debe vivir en la laptop) y arrancó cosas de Lumina que no tienen sentido en el Jetson. Resultado útil: se leyó el código y se comprobó que **ningún botón de la celda puede mover un pin** mientras `gpiod` no esté instalado (capítulo 7).

Caso 7: ningún navegador abría (21 de octubre)

Síntoma: Los navegadores instalados como **snap** (paquetes autocontenidos de Ubuntu; por ejemplo Chromium) fallaban con `snap-confine ... required permitted capability cap_dac_override not found`. **Hipótesis de fuentes externas:** una versión nueva de `snapd` en un kernel sin ciertas opciones. Aquí hay **dos `snapd`** (el `.deb` 2.67.1 y el snap 2.77.1). La causa raíz **no está diagnosticada**. **Solución parcial:** se instaló Firefox desde el **PPA de Mozilla** (repositorio oficial) como `.deb`, versión `158.0.1~mt1`, con `--allow-`

`downgrades`. **Abre** el dashboard de la laptop, pero **va lento por VNC** (≈ 87 % de un núcleo en `firefox`): se interpreta como renderizado por software a través de VNC; **no está medido con un monitor HDMI**. **Efecto en Lumina:** `robot.html` usa `SpeechRecognition` del navegador, que no está activo en Firefox; **por eso la voz de Lumina quedó en la laptop** (ADR 0012, capítulo 15).

Caso 8: la cámara que el navegador no mostraba (23 de octubre)

El agente del Jetson funcionó: **29.0 FPS** de cámara, **29.1** de entrega a 640 × 480, inferencia **68.0 ms** de promedio, `laptop.reachable` verdadero, `/stream` con respuesta 200 en `127.0.0.1:8001`. Pero la página **no mostró el video**: Firefox bloqueó la imagen por la **política de seguridad (CSP)**. La causa y la corrección están en el capítulo 11 (web); aquí queda como ejemplo de que **el agente puede estar perfecto y la pantalla vacía**, y de que hay que diagnosticar por capas.

Caso 9: el audio de Lumina bloqueado (23 de octubre)

Otro bloqueo del mismo tipo, esta vez con el **audio** (`media-src`). Capítulo 11.

La regla de seguridad del Jetson: nunca `gpiod`

Tu usuario en el Jetson pertenece al grupo `gpio`. `cell_line.py` hace `import gpiod` al cargarse y, **si existe**, configuraría como **salidas** las líneas 8 y 9 de `/dev/gpiochip0`: números **BCM de Raspberry Pi**, que en el **SoC del Jetson** (el chip) corresponden a **otras líneas**. Por eso:

- **No** se instala `gpiod` de Python en el Jetson (**VERIFICADO**: `python -c "import gpiod"` falla; sí están las herramientas del sistema `gpiod` 1.6.3 y `libgpiod2`).
- El script de arranque del agente **se niega a seguir** si lo detecta.
- Antes de arrancar cualquier backend se comprueba:

```
# [Jetson · SSH], dentro del entorno virtual
python -c "import gpiod"      # DEBE fallar con ModuleNotFoundError
```

Cómo se arranca el agente del Jetson

Todo desde tu laptop, por **SSH**, o directo en el Jetson. Las rutas y nombres vienen de `COMANDOS_LINUX.md`, sección 8 (**construido y probado solo en la laptop; los scripts `.sh` no se han corrido en el Jetson**, salvo la prueba reportada del 23 de octubre).

```
# Una vez: configuración (edita AETHER_LAPTOP_URL con la IP de la laptop)
[Jetson · SSH]
cp ~/aether/scripts/jetson/aether-agent.env.example ~/.aether-agent.env && nano
~/.aether-agent.env

# Arrancar el agente dentro de tmux (sesión "agente")
[Jetson · SSH]
bash ~/aether/scripts/jetson/iniciar-agente.sh

# Medir: FPS de cámara y de entrega, ms de inferencia, CPU y RAM
[Jetson · SSH]
python3 ~/aether/scripts/probar_agente.py http://localhost:8001 --segundos 15

# Detener
[Jetson · SSH]
bash ~/aether/scripts/jetson/detener-agente.sh
```

`iniciar-agente.sh` hace varias comprobaciones antes de arrancar (que exista `tmux`, el entorno virtual, `/dev/video0`, el archivo de configuración, y que `gpiod` **no** esté), dice **qué falta y cómo arreglarlo**, y se puede repetir sin miedo: si el agente ya corre, lo dice y no hace nada. El log queda en `~/aether-agente.log`. Para ver la página de la laptop **dentro del Jetson** (por VNC): `bash ~/aether/scripts/ver-aether.sh robot.html`.

Las dos IP de la red del laboratorio

Según la bitácora, la red de esos días fue `192.168.100.x`: el Jetson por cable en `192.168.100.19` y la laptop por WiFi en `192.168.100.14`. El Jetson alcanzó el backend de la laptop (`curl http://192.168.100.14:8000/health` devolvió `{"status":"ok"}`); antes dio *timeout* simplemente porque el backend no estaba corriendo. Esas direcciones **cambian con la red**, por eso `agent_config.py` admite **nombres** (`mario-desktop.local`) y las compara con las IP a las que apuntan (con una caché de 60 s). La resolución real del nombre depende de la red y **no está verificada**.

Qué falta medir, en una frase

La cifra que más falta es la de **FPS con buena luz, la cámara en su lugar y el modo de energía anotado** (**PENDIENTE (P-24)**): el objetivo de **≥ 30 FPS** es de diseño y la prueba de la laptop (30.0 FPS con webcam y YOLO en CPU) no dice nada del Jetson. Tampoco se ha probado el modo `MAXN_SUPER`, ni TensorRT (se decidirá con datos), ni cómo se alimentará el Jetson dentro del robot (capítulo 4). Los diagnósticos que quedaron abiertos

(cortes de SSH, `snap-confine`, propiedad de `libcusparseLt`) están en la lista de pendientes del final del libro (**P-28**), junto con la duda de si el robot tendrá micrófono (**P-22**).

Qué hace la industria

La línea **Jetson** de NVIDIA es la plataforma típica de robots autónomos pequeños, cámaras inteligentes y drones: percepción en el borde (*edge*), sin nube, con consumo de unos pocos vatios. En producción se usan **imágenes de sistema ya empaquetadas**, contenedores (Docker) para reproducir el entorno y **TensorRT** para exprimir el rendimiento. Tu proyecto sigue el camino del laboratorio (venv + `requirements`), que es más fácil de entender, con una tabla de versiones reportadas como la de arriba, y deja Docker y TensorRT como pasos de madurez.

Para ponerte a prueba

1. ¿Qué dos cosas distintas protege `YOLO_AUTOINSTALL=False` y el `--no-deps` al instalar `ultralytics`?
2. ¿Por qué el torch de PyPI y el torch de NVIDIA no son lo mismo aunque se llamen igual?
3. Si `torch.cuda.is_available()` devolviera `False`, ¿qué tres cosas revisarías primero según la bitácora?
4. ¿Por qué una cifra de FPS necesita decir en qué modo de energía se midió?
5. ¿Por qué el agente puede funcionar bien y aun así no verse el video en la página?
6. ¿Por qué nunca debe instalarse `gpiod` en el Jetson?

Glosario de siglas y términos

Cada término se explica la primera vez que aparece; aquí está todo junto. La columna «Cap.» dice dónde se explica. (Esta versión cubre los capítulos 2 a 9 y crece con cada parte.)

Término	Qué significa	Cap.
A4988	Driver (circuito de potencia) para motores a pasos: recibe STEP y DIR y entrega hasta 1.0 A por bobina.	5
ADR	Architecture Decision Record: documento corto que registra una decisión de arquitectura, sus alternativas y su estado.	3, 7
AGV / AMR	Vehículo guiado automáticamente / robot móvil autónomo: robots que mueven carga en almacenes.	3, 6
Amperio (A)	Unidad de corriente eléctrica: cuánta carga pasa por segundo.	2
Amperios-hora (Ah)	Capacidad de una batería: corriente por tiempo que puede entregar.	4
ArUco	Cuadro blanco y negro con patrón único que una cámara usa para saber dónde está y hacia dónde mira.	6
AWG	American Wire Gauge: número de calibre de un cable; menor número = cable más grueso.	4
Banda transportadora	Cinta que lleva las piezas por la celda.	3, 7
BCM	Numeración lógica de los pines GPIO de la Raspberry (por el chip de Broadcom), distinta del número físico en el conector.	8
BNO055	Módulo IMU (acelerómetro, giroscopio y magnetómetro) con dirección I^2C 0x28.	6
Bornera KF301	Conector atornillable para cables, soldado a la placa.	4
Bus	Conjunto de cables compartidos por varios dispositivos (I^2C, RS485).	5, 6
CC / CA (DC / AC)	Corriente continua (siempre en el mismo sentido) y corriente alterna (cambia de sentido; la de la pared).	2
Centro de gravedad	Punto donde parece concentrarse todo el peso; cuanto más alto, más fácil volcar.	3
Cortocircuito	Unión accidental entre dos puntos de distinto voltaje que deja pasar una corriente enorme.	4
CSP	Content Security Policy: la política con que el navegador decide qué recursos puede cargar una página.	9
CUDA	Herramientas de NVIDIA para que los programas usen la GPU.	9
cuDNN	Biblioteca de NVIDIA con operaciones de redes neuronales sobre CUDA.	9

Término	Qué significa	Cap.
Diferencial (robot)	Robot con dos motores y una llanta por motor; gira cambiando la velocidad relativa de las llantas.	3, 5
Driver	Circuito que obedece una señal débil y entrega la corriente grande que necesita un motor.	2, 5
ECU	Engine Control Unit: computadora de motor de un coche; es la pieza que viaja en cada fixture.	3, 7
Electroválvula	Válvula que abre o cierra el aire cuando recibe corriente; mueve el pistón.	7, 8
Encoder	Sensor que cuenta cuánto gira un eje.	3, 5
ESP32	Microcontrolador con Wi-Fi y Bluetooth; en Aether es el controlador de bajo nivel del robot.	5
Final de carrera	Interruptor que se activa cuando una pieza móvil llega a su límite; da el cero (home).	3, 5, 6
Fixture	Base que sostiene y posiciona cada pieza sobre la banda y dentro del carrito.	3, 7
FPS	Frames per second: cuadros por segundo que entrega una cámara.	6, 9
Fusible	Hilo calibrado que se funde si pasa más corriente de la permitida y corta el circuito.	2, 4
GND	Ground: tierra o referencia de 0 V contra la que se miden los voltajes.	2, 4
GPIO	General Purpose Input/Output: pin que puede leer o encender una señal.	2, 8
GPU	Graphics Processing Unit: chip con miles de núcleos pequeños; acelera redes neuronales.	9
HMI	Human-Machine Interface: pantalla con la que un operador maneja una máquina.	7
Home	Posición cero de un mecanismo, que se busca con un final de carrera.	3, 5
IMU	Inertial Measurement Unit: unidad que mide giro y aceleración.	6
Infrarrojo industrial	Sensor de 3 hilos (+24 V, 0 V y señal) que detecta presencia; puede ser PNP o NPN.	2, 7
I^2C	Bus de dos cables (SDA y SCL) donde varios dispositivos comparten línea, cada uno con su dirección.	6
JetPack / L4T	Sistema y controladores de NVIDIA para Jetson (Linux for Tegra).	9
KCD1	Interruptor de encendido de la rama de lógica del carrito.	4
LAY37	Modelo del botón de paro de emergencia del carrito (contacto normalmente cerrado).	4
LOGO	Mini controlador lógico programable; en la celda mueve la banda.	7

Término	Qué significa	Cap.
MAX485	Chip que traduce el serial del ESP32 a RS485.	5
Microcontrolador	Chip que ejecuta un solo programa sin sistema operativo, con pines que controla con precisión.	5
Micropaso	Subdivisión de un paso de un motor a pasos; 1/16 da 3200 pulsos por vuelta.	5
Modbus TCP	Protocolo industrial para leer y escribir registros de un PLC por Ethernet (puerto 502).	7
Multímetro	Instrumento para medir voltaje, corriente y continuidad.	2
NEMA 17	Tamaño estándar (42 × 42 mm) de motor a pasos; aquí el 17HS4401.	3, 5
Newton-metro (N·m)	Unidad de torque: fuerza que hace girar.	2
NO / NC	Contacto normalmente abierto / normalmente cerrado.	4
Odometría	Estimar la posición sumando cuánto giró cada rueda.	5
Ohmio (Ω) y ley de Ohm	Unidad de resistencia; V = I × R.	2
ONNX	Formato neutral para guardar modelos de redes neuronales.	8
Optoacoplador	Chip que pasa una señal con luz para separar eléctricamente dos circuitos.	2, 8
PLC	Programmable Logic Controller: controlador industrial robusto para tableros.	7
PNP / NPN	Tipos de salida de un sensor de 3 hilos: entrega +24 V o conecta a 0 V al detectar.	2
Potencia (W)	Trabajo por segundo; P = V × I.	2
PWM	Pulse-Width Modulation: encender y apagar muy rápido y variar la proporción para controlar potencia.	5
QR / código de barras	Códigos que una cámara lee; identifican cada fixture.	6
QTR-8A	Barra de 8 sensores infrarrojos que ven el piso (seguidor de línea, respaldo).	6
Relé / relevador	Interruptor que se cierra al energizar una bobina; separa el mundo de 3.3 V del de potencia.	2, 4, 8
RS485	Estándar de comunicación de dos cables trenzados (A y B), robusto para entornos con ruido.	5
SDA / SCL	Cables de datos y de reloj del bus I^2C.	6
Servo	Motor pequeño que gira a un ángulo indicado por una señal PWM (MG90S).	5

Término	Qué significa	Cap.
STEP / DIR / ENABLE	Señales de un driver de pasos: un pulso por paso, el sentido y el activar/soltar.	5
Tierra común	Unir los GND de todas las placas para que compartan referencia.	2, 4
Time-of-Flight (ToF)	Medir distancia por el tiempo que tarda un pulso de luz en rebotar.	6
TR8	Husillo (tornillo de potencia) de 8 mm.	3, 5
USB UVC	Estándar de cámaras USB que Linux reconoce sin controladores extra.	6
V4L2	Video4Linux2: controlador estándar de cámaras en Linux.	6
VL53L0X	Sensor de distancia de tiempo de vuelo.	6
Voltio (V)	Unidad de voltaje: la «presión» eléctrica.	2
Vref	Voltaje de referencia de un A4988 que fija la corriente máxima por bobina.	4
Watchdog	Perro guardián: temporizador que detiene el robot si el programa deja de mandar su latido.	4, 5
XL4005	Módulo reductor de voltaje (12 → 5 V).	4
XSHUT	Pin de apagado de un VL53L0X; sirve para darles direcciones distintas.	6
XT60	Conector amarillo de potencia para baterías.	4

Pendientes de confirmar con el equipo

Todo lo que el libro dejó marcado como PENDIENTE (P-01, P-02, …) está aquí, y solo aquí. Cada línea dice qué falta, con quién se confirma y en qué capítulo aparece. Mientras no se confirme, el libro lo trata como "no sabemos".

Código	Qué falta	Con quién	Capítulos
P-01	Motores definitivos del robot: ¿M0601 (FIT1042 / FIT1038) u otros? Voltaje, corriente y torque	Mario, David, Isaac	2, 3, 5
P-02	Peso total real, reparto por llanta (hoy: máx. 40 kg y ~17 kg por llanta motriz), altura del centro de gravedad y pendiente máxima del piso	David, Isaac	2, 3
P-03	Diámetro real de las llantas motrices y de las rodajas	David, Isaac	2
P-04	Estado real del armado del carrito; si la hoja rev. 3.8 es el diseño que se construye; material, impresora y relleno de las piezas impresas	Isaac, David	3
P-05	Modelo y capacidad reales de la batería (la hoja dice BR-1224 de 12 V, 24 Ah)	David	3, 4
P-06	Paro de emergencia del robot: ¿el software debe enterarse (pin del ESP32)? Modelo, cableado y ubicación definitivos	David, Mario	4, 5
P-07	Con qué software se programa el ESP32 (Arduino IDE, PlatformIO, ESP-IDF o MicroPython)	Mario	5
P-08	Modelo exacto del ESP32 en mano (la hoja supone un DevKitC de 38 pines)	Mario	5
P-09	Protocolo de mensajes RS485 del motor-rueda que se compre	Mario, David	5
P-10	Paso real (avance por vuelta) del husillo TR8: define todo el programa del elevador	David, Isaac	5
P-11	Sensores definitivos: ¿habrá lidar? ¿VL53L0X × 4, BNO055 y QTR-8A confirmados?	Mario, David	6
P-12	Tamaño impreso, colocación y distancia de lectura de los ArUco	Mario, Isaac	6
P-13	Cableado real de la celda: GPIO + LOGO o PLC por Modbus	Uriel, David, Pedro	7, 8
P-14	Marca y modelo del LOGO o PLC y de los dos infrarrojos (PNP/NPN), y sus bornes	Uriel, David	2, 7
P-15	Pistón: tipo, electroválvula, voltaje de la bobina, presión y tiempos reales	Pedro	7
P-16	Banda: motor, velocidad real, altura de la cinta, si corre sola o la arranca el LOGO	Isaac, David, Pedro	3, 7

Código	Qué falta	Con quién	Capítulos
P-17	Paro de emergencia físico de la celda y arranque/paro físicos	David, Pedro	4, 7, 8
P-18	Placa de relevadores: activa en alto o en bajo, aislamiento, diodo y alimentación	David, Uriel	8
P-19	Alcance de la HMI de la celda (pantalla propia, quién la opera)	Emmanuel	7
P-20	Pruebas de la Pi 4 en hardware real: sistema, RAM, versión de OpenCV, CPU, cámara	Mario, Uriel	7, 8
P-21	Alimentación del Jetson en el robot y cómo se evita el doble camino de 5 V hacia el ESP32	David	4, 9
P-22	Micrófono y bocina en el robot o en el Jetson	Mario	6, 9
P-23	Dimensiones y material oficiales de los fixtures y quién los diseña	Isaac	7
P-24	FPS, exposición y calidad de imagen del Jetson con buena luz, cámara en su lugar y modo de energía anotado	Mario	6, 9
P-25	Contrato de mensajes entre el Jetson y el ESP32 (mensajes, latido, tiempo de frenado)	Mario	5
P-26	Criterio real de PASS/FAIL por defectos (hoy solo reconoce piezas registradas)	Mario	7
P-27	Arranque automático de la celda al encender la Pi (systemd u otro)	Mario, Uriel	8
P-28	Diagnósticos abiertos del Jetson: causa de los cortes de SSH, de `snap-confine` y propiedad de `libcusparseLt`	Mario	9

Este listado crece con cada parte del libro.